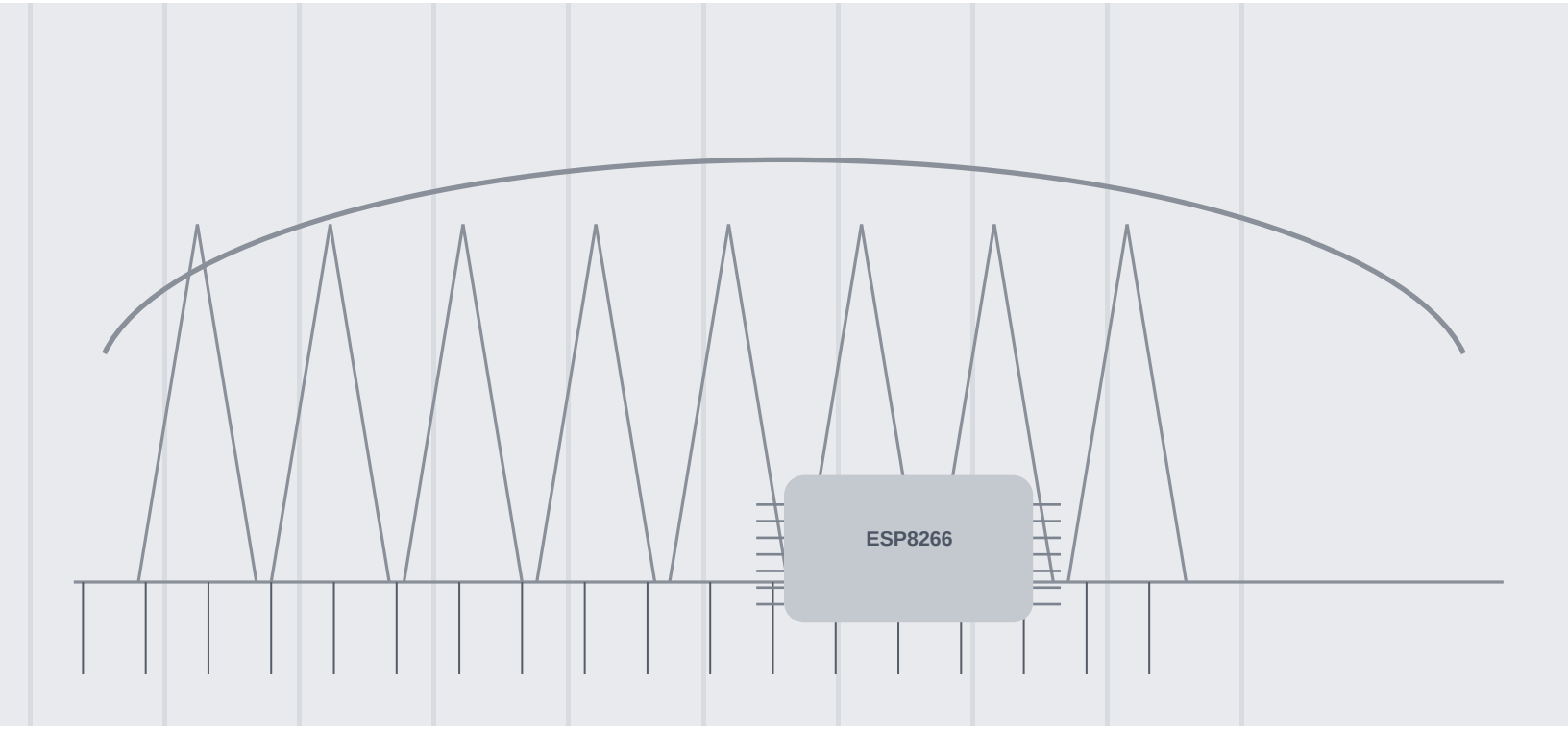
A practical guide to

KernelESP

microcontrollers, ESP8266 and tiny UNIX automation



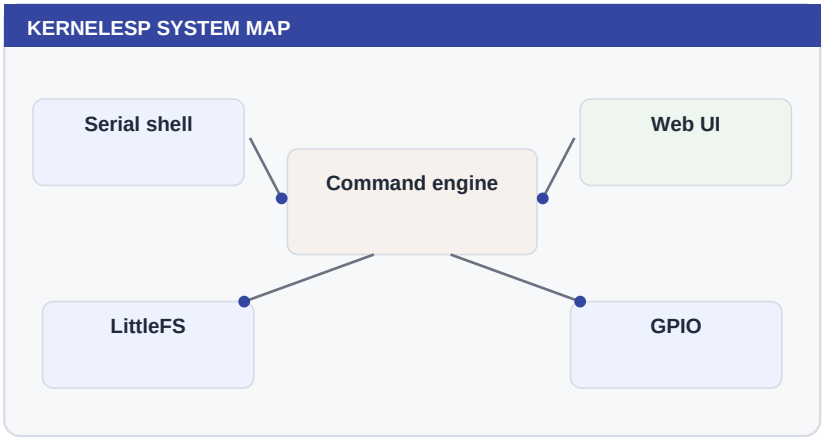
Serial shell - Web UI - automation language - relays - sensors - garden projects - recovery

KernelESP Beginner's Book

This book is written for people who are new to microcontrollers, new to embedded web interfaces, or simply want a calm path into KernelESP. It treats the ESP8266 as a tiny UNIX-like machine: it has commands, files, logs, scripts, cron jobs, relays, sensors, inputs, mail alerts and a web interface that keeps everything visible.

How to read it

Start at Part 1 if the board is new to you. Jump to the projects in Part 8 when you want irrigation, alarms or scheduled actions. Keep Part 10 nearby when something goes wrong.



About this edition

This generated edition is part of the KernelESP repository. It is not a replacement for the command reference; it is a guided tour that explains why each feature exists, when to use it and how to test it safely on real hardware.

The examples prefer 24-hour time, non-blocking timers and explicit recovery paths. That style matters on a small ESP8266: clarity is a safety feature, and short commands keep the serial console and web UI responsive.

Try it

```
help
health
free
wifi status
crontab -l
relay status
```

Table of Contents

This index is clickable in PDF viewers that support internal links. Use it like a control panel for the book.

The Learning Map

Every chapter pair follows the same rhythm: first a plain-English explanation, then a lab page with commands, checks and a safe next step.

The method

- Inspect before changing: health, logs and status commands are your first tools.
- Prefer non-blocking actions: timers and cron are safer than long sleeps.
- Make names human: trees, flowers and fan are better than relay1, relay2 and gpio5.
- Keep recovery nearby: serial, backups and disarm are part of the design.

How to practice safely

For each idea, start with a read-only command. Then try a harmless command such as logger or echo. Only after that should you connect real hardware. This mirrors the style used in professional administration books: concept, procedure, verification and recovery.

The KernelESP loop

Understand the concept, run the smallest useful command, inspect the result, save only what you can explain, and keep a way back.

Your learning goal

Which first project do you want to build: garden, fan, heater, alarm or dashboard?
What is the safest harmless output you can test before controlling real hardware?

PART 1

Microcontroller Foundations

This part covers What Is a Microcontroller?, Meet the ESP8266, Microcontroller vs Computer and builds toward practical confidence.

Chapters

How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

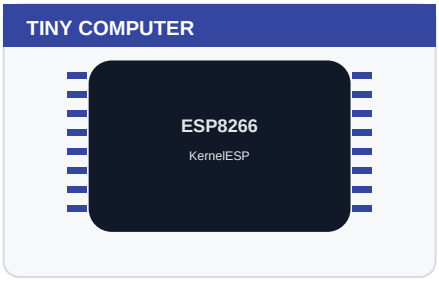
Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

1. What Is a Microcontroller?

A microcontroller is a tiny computer built to control devices rather than to replace a desktop or laptop.

A microcontroller is a small computer designed to live inside a device. It usually has a processor, memory and input/output pins in one package. It does not try to be a laptop. It tries to read signals, make decisions and control things reliably.



The mental model

Think of it as a very focused worker: it wakes up, reads inputs, follows firmware and changes outputs. It is excellent at simple repeated jobs.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- MCU: microcontroller unit, the small computer inside the device.
- GPIO: a pin that can read or switch simple electrical signals.
- Firmware: the program stored on the chip.

Why this matters

This concept explains why KernelESP is powerful but also why it must respect memory, power and timing limits.

Common beginner mistake

Expecting a microcontroller to behave like a full PC with unlimited memory, storage and multitasking.

One-minute review

Explain what is a microcontroller? in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 1: What Is a Microcontroller?

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
uname
chip
flash
free
board show
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Point to one device in your house that probably contains a microcontroller and describe what it senses or controls.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

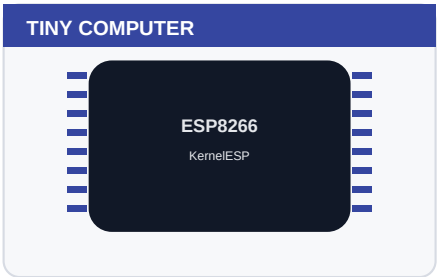
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

2. Meet the ESP8266

The ESP8266 is a Wi-Fi microcontroller that can run KernelESP, host a web UI and control real-world devices.

The ESP8266 is a popular Wi-Fi microcontroller. It is small, inexpensive and powerful enough to host a web page, talk to sensors and switch relays, but it still has tight memory limits. KernelESP is designed around those limits.



The mental model

It is small enough to hide in an enclosure but capable enough to become a local web-connected controller.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- MCU: microcontroller unit, the small computer inside the device.
- GPIO: a pin that can read or switch simple electrical signals.
- Firmware: the program stored on the chip.

Why this matters

Knowing the ESP8266 makes later design choices feel less mysterious: every feature shares the same small memory and processor.

Common beginner mistake

Judging the ESP8266 by its price. It is inexpensive, but not disposable when it controls water, heat or alarms.

One-minute review

Explain meet the esp8266 in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 2: Meet the ESP8266

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
chip
flash
wifi mac
free
sysinfo
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Write down the chip, flash and free heap values of your board as its identity card.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

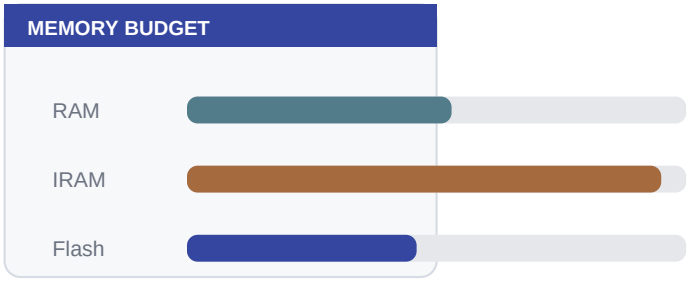
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

3. Microcontroller vs Computer

A microcontroller and a normal computer are both programmable, but they are built for very different jobs.

A normal computer is built for many programs, large storage, a screen, a keyboard and constant interaction. A microcontroller is built for a focused job: watch a few inputs, control a few outputs and keep going after power cuts.



The mental model

A computer is a general-purpose workstation. A microcontroller is embedded inside a product and usually runs one carefully designed firmware.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

This difference helps beginners understand why KernelESP favors short commands, simple files and non-blocking timers.

Common beginner mistake

Trying to install desktop-style software or heavy scripting languages on a tiny embedded board.

One-minute review

Explain microcontroller vs computer in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 3: Microcontroller vs Computer

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
ps
top
df
uptime
health
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Compare the output of free and df with the memory and disk of your laptop.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

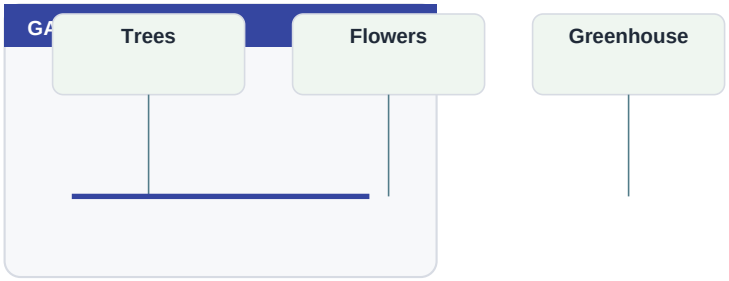
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

4. What Can You Build?

KernelESP can control gardens, heating helpers, fans, alerts, sensors, timed routines and local dashboards.

The useful question is not only what the chip can do, but what the chip can observe and control. With relays, inputs and sensors, KernelESP can manage watering zones, fans, heaters, warning lights, alarms and daily status reports.



The mental model

The pattern is always the same: observe something, decide something, act on something and log what happened.

Where you will use it

- Control tree, flower or greenhouse watering zones with solenoid valves.
- Skip watering when rain, heat or time windows make irrigation unsafe.
- Log every watering decision so you can troubleshoot dry plants or wasted water.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Projects become much easier when you map them as sensor plus decision plus actuator plus recovery path.

Common beginner mistake

Starting with a big project before proving one input, one output and one schedule.

One-minute review

Explain what can you build? in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 4: What Can You Build?

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay status
sensor read
crontab -l
input list
mail status
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Choose one realistic project and split it into inputs, decisions, outputs and safety checks.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

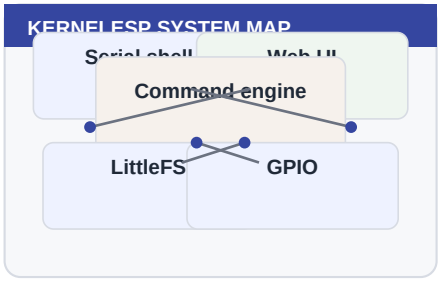
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

5. How KernelESP Changes the Board

KernelESP gives the ESP8266 a small UNIX-like personality: commands, files, logs, scripts, cron and a web UI.

A bare ESP8266 runs firmware. KernelESP adds a vocabulary: commands, files, cron, scripts, a web UI and a tiny C-like automation language. That vocabulary makes the board teachable and inspectable.



The mental model

Instead of recompiling for every tiny change, you can inspect and configure the board from serial, web or API.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

The shell makes the board teachable. You can learn by asking it what it knows.

Common beginner mistake

Thinking KernelESP hides the hardware. It actually exposes the hardware through safer, named commands.

One-minute review

Explain how kernelesp changes the board in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 5: How KernelESP Changes the Board

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
help
ls /
cat /etc/config.txt
history
diag
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Run help and ls /, then explain why this feels more like a tiny server than a bare microcontroller.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

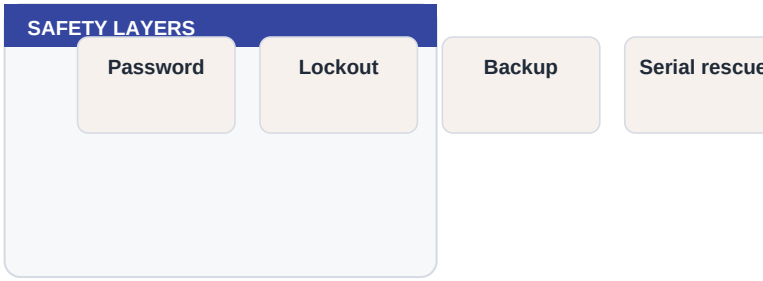
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

6. Beginner Safety and Mindset

The safest automation is built in small steps: simulate, log, test low voltage, then connect real equipment.

The safest beginner path is slow and observable. First log, then dry run, then test a low-voltage load, and only then connect real actuators. Good automation is not magic; it is a chain of small verified steps.



The mental model

A careful beginner is not slow; a careful beginner is building evidence.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

This habit matters because KernelESP may control water, fans, heaters or alarms in the physical world.

Common beginner mistake

Treating software tests and hardware tests as the same thing. Hardware tests need extra caution.

One-minute review

Explain beginner safety and mindset in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 6: Beginner Safety and Mindset

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
dryrun on
logger first_test
health
dryrun off
backup
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Before any real relay test, say what command will stop the action.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

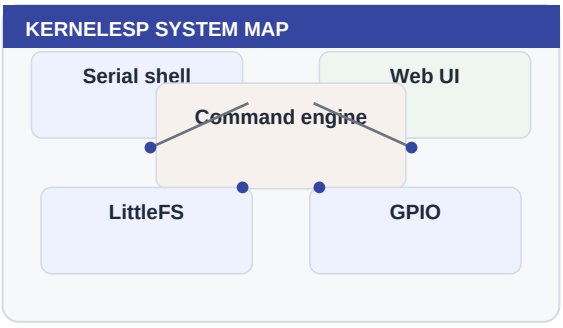
What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

PART 2

Orientation

This part covers Meet the Tiny UNIX, Serial Console First Contact, Find the Web Interface and builds toward practical confidence.

Chapters



How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

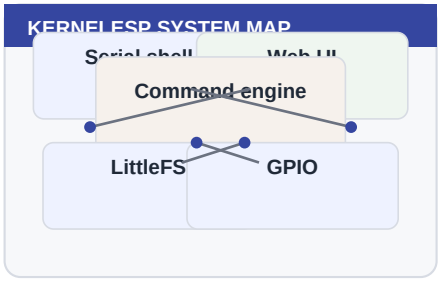
Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

7. Meet the Tiny UNIX

KernelESP turns an ESP8266 into a small networked computer with a shell, files, web pages and automation.

System commands are your dashboard. They answer what the board is, what it is doing and whether it has enough resources to continue.



The mental model

Think of the board as a tiny server. It cannot run Linux, but it borrows the useful habits: inspect first, change second, save the result.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

A mental model prevents panic. If it has commands, files and logs, you can reason about it.

Common beginner mistake

Treating the ESP8266 like a black box. KernelESP is inspectable; use that gift.

One-minute review

Explain meet the tiny unix in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 7: Meet the Tiny UNIX

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
uname
version
whoami
help
health
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Run help for three commands you do not understand yet and write down what each one controls.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

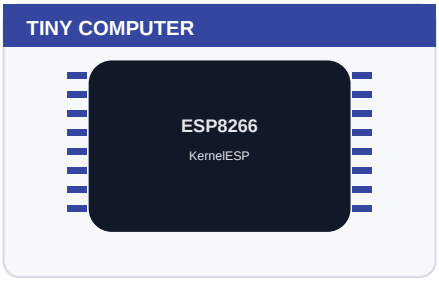
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

8. Serial Console First Contact

The serial console is the most reliable way to talk to the board, especially during Wi-Fi trouble.

The chip is small, but not mysterious. If you can read status, inspect files and run small commands, you can understand what it is doing.



The mental model

Serial is the local keyboard and screen. It works before the network is ready and remains the best rescue route.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- MCU: microcontroller unit, the small computer inside the device.
- GPIO: a pin that can read or switch simple electrical signals.
- Firmware: the program stored on the chip.

Why this matters

Every serious installation should keep a serial rescue path available during setup.

Common beginner mistake

Changing Wi-Fi and then closing the only working console.

One-minute review

Explain serial console first contact in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 8: Serial Console First Contact

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
help
wifi status
free
df
dmesg
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Unplug Wi-Fi mentally: which commands would still help you recover?

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

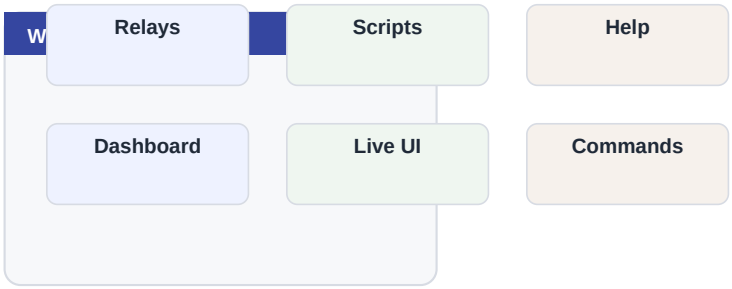
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

9. Find the Web Interface

The web UI gives beginners a friendly control panel for commands, relays, scripts, logs and help.

The web UI is a friendly window into KernelESP. It should make common work easier, but the underlying commands remain the source of truth.



The mental model

The browser is a remote window into the same command engine used by serial.

Where you will use it

- Run commands without opening a serial terminal.
- Edit scripts from a browser.
- Read local help when the internet is unavailable.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Seeing the same state in two places builds confidence that you are controlling the right board.

Common beginner mistake

Assuming the browser is broken when the board simply changed IP.

One-minute review

Explain find the web interface in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 9: Find the Web Interface

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
wifi ip
wifi status
hostname
health
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Open the IP address in a browser and compare Dashboard data with health output.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

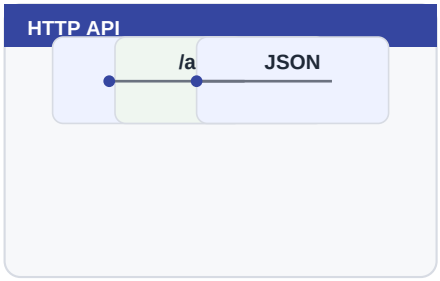
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

10. Use the HTTP API

The API lets scripts and other computers ask KernelESP to run commands and return JSON.

The API is a bridge to other computers. Start with read-only status calls, then add control only when local safety and logging are already in place.



The mental model

The API is the command line with a URL around it. It is useful for integrations and tests.

Where you will use it

- Read status from a laptop or home dashboard.
- Trigger a command from another local automation system.
- Build tests that verify the board is alive after updates.

Terms to remember

- Endpoint: a URL that performs a specific task.
- JSON: structured text returned by the API.
- URL encoding: safe representation of spaces and symbols.

Why this matters

APIs are how small devices join bigger systems without needing a cloud service.

Common beginner mistake

Forgetting URL encoding when a command contains spaces, quotes or symbols.

One-minute review

Explain use the http api in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 10: Use the HTTP API

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
curl -G http://<ip>/api/cmd --data-urlencode key=admin --data-urlencode c=free
curl http://<ip>/api/status?key=admin
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Run an API command from your laptop and compare it with the web command runner.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

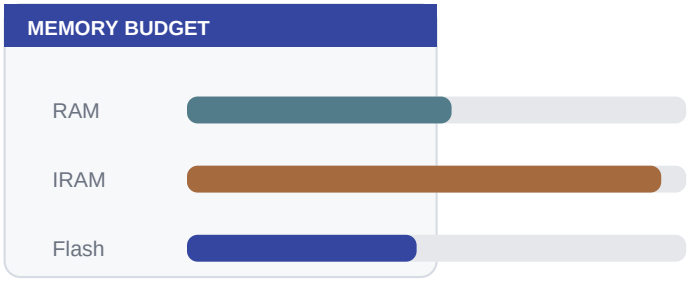
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

11. Read System Health

Health commands tell you whether the board is stable before you add automation.

Memory is a budget. KernelESP can do a lot because it keeps features small and avoids long blocking work.



The mental model

Heap, Wi-Fi state, filesystem space and time are vital signs.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

A baseline makes future debugging factual instead of emotional.

Common beginner mistake

Ignoring low heap or high fragmentation until the web UI feels slow.

One-minute review

Explain read system health in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 11: Read System Health

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
health
free
uptime
df
sysinfo
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Record heap before and after opening Live UI, then after running a script.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

PART 3

Files and Shell Confidence

This part covers Understand LittleFS, Create and Inspect Files, Use Pipes Like a Pro and builds toward practical confidence.

Chapters

LITTLEFS LAYOUT

```
/
  /etc     config, cron, relays
  /home    scripts and notes
  /www     web interface
  /var/log kernel.log
  /func    persistent functions
```

How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

12. Understand LittleFS

KernelESP stores configuration, scripts and web assets in LittleFS.

Files make the board persistent. A command changes the current moment; a file lets the board remember a script, configuration or note after reboot.

LITTLEFS LAYOUT

```
/
  /etc      config, cron, relays
  /home     scripts and notes
  /www      web interface
  /var/log  kernel.log
  /func     persistent functions
```

The mental model

LittleFS is the board's small disk. It survives reboot but has limited space.

Where you will use it

- Store scripts, installation notes and boot commands.
- Inspect configuration without recompiling firmware.
- Back up the board before experiments.

Terms to remember

- LittleFS: the small flash filesystem used by KernelESP.
- /etc: persistent configuration.
- /home: user scripts and notes.

Why this matters

Files make automations persistent and inspectable.

Common beginner mistake

Saving everything in one giant file instead of small named files.

One-minute review

Explain understand littlefs in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 12: Understand LittleFS

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
ls /
ls /etc
ls /home
df
du /
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Create a tiny note in /home, read it, then delete it.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

13. Create and Inspect Files

The shell can create, append, read, search and remove files.

Files make the board persistent. A command changes the current moment; a file lets the board remember a script, configuration or note after reboot.

LITTLEFS LAYOUT

```
/
  /etc      config, cron, relays
  /home     scripts and notes
  /www      web interface
  /var/log  kernel.log
  /func     persistent functions
```

The mental model

File commands are small building blocks for scripts and backups.

Where you will use it

- Store scripts, installation notes and boot commands.
- Inspect configuration without recompiling firmware.
- Back up the board before experiments.

Terms to remember

- LittleFS: the small flash filesystem used by KernelESP.
- /etc: persistent configuration.
- /home: user scripts and notes.

Why this matters

A beginner who can inspect files can recover from many configuration mistakes.

Common beginner mistake

Editing a file without reading it first.

One-minute review

Explain create and inspect files in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 13: Create and Inspect Files

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
write /home/hello.txt hello
append /home/hello.txt world
cat /home/hello.txt
grep hello /home/hello.txt
rm /home/hello.txt
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Create a checklist file for your installation.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

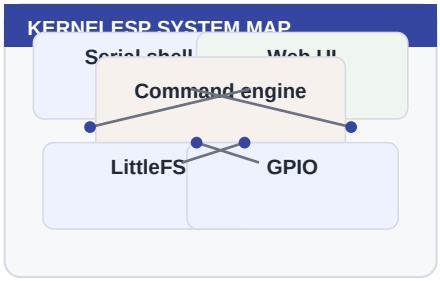
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

14. Use Pipes Like a Pro

Pipes connect small commands so one command filters another.

System commands are your dashboard. They answer what the board is, what it is doing and whether it has enough resources to continue.



The mental model

A pipe is a little conveyor belt: left command produces text, right command narrows it down.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Pipes make the tiny shell feel surprisingly UNIX-like.

Common beginner mistake

Using pipes for everything; sometimes one direct command is clearer.

One-minute review

Explain use pipes like a pro in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 14: Use Pipes Like a Pro

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
health | grep wifi
dmesg | tail -n 5
cat /var/log/kernel.log | grep cron
health | wc -l
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Find all log lines containing wifi or ntp.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

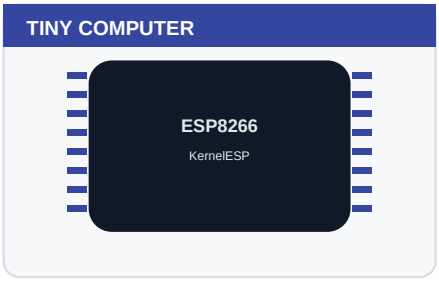
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

15. History and Aliases

History and aliases save typing during repeated setup work.

The chip is small, but not mysterious. If you can read status, inspect files and run small commands, you can understand what it is doing.



The mental model

An alias is a short nickname for a longer command.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- MCU: microcontroller unit, the small computer inside the device.
- GPIO: a pin that can read or switch simple electrical signals.
- Firmware: the program stored on the chip.

Why this matters

Shortcuts reduce mistakes when commands are long.

Common beginner mistake

Making aliases with names that hide real commands.

One-minute review

Explain history and aliases in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 15: History and Aliases

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
history
alias ll ls /
ll
alias save
history save
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Create an alias for health and remove it later.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

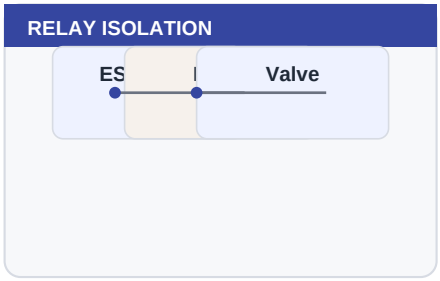
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

16. Dry Run Mode

Dry run helps test dangerous commands without energizing hardware.

Hardware control is where software becomes physical. Treat every relay command as a real-world action, even while testing, and prefer names that describe the device being controlled.



The mental model

It is a rehearsal mode for actuators.

Where you will use it

- Switch a fan, warning lamp, low-voltage valve or contactor input.
- Pulse a test load briefly before scheduling longer work.
- Keep relay names tied to physical labels on the enclosure.

Terms to remember

- Relay: an electrically controlled switch.
- Active-low: the relay turns on when the GPIO goes low.
- External supply: power source for relay coils or valves.

Why this matters

Safety-first habits matter more than clever automation.

Common beginner mistake

Testing new relay logic directly on live hardware.

One-minute review

Explain dry run mode in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 16: Dry Run Mode

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
dryrun on
relay on valve1
relay status
dryrun off
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Use dryrun before connecting a pump, valve or mains relay.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

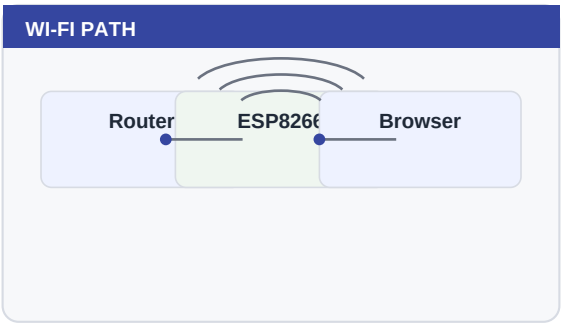
What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

PART 4

Networking and Time

This part covers Scan and Connect Wi-Fi, Save Wi-Fi Profiles, Recover Wi-Fi and builds toward practical confidence.

Chapters



How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

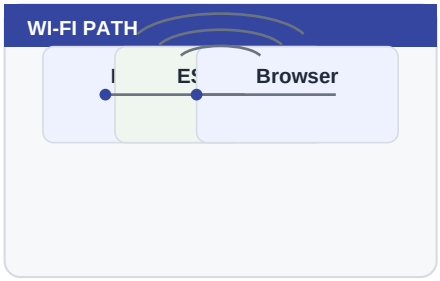
Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

17. Scan and Connect Wi-Fi

Wi-Fi commands connect the board while keeping the shell responsive.

Network features are a convenience layer over the same command engine. Use the serial console as the ground truth, then use the browser and API once the board is reachable.



The mental model

Connection starts in the background; you keep control while it negotiates.

Where you will use it

- Move the board from bench Wi-Fi to garden Wi-Fi.
- Find the IP address for the web UI.
- Recover access after a router or password change.

Terms to remember

- SSID: the Wi-Fi network name.
- DHCP: automatic IP address assignment by the router.
- Static IP: a manually chosen address for predictable access.

Why this matters

Non-blocking Wi-Fi avoids freezing the rescue console.

Common beginner mistake

Running many network commands while the first connection is still settling.

One-minute review

Explain scan and connect wi-fi in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 17: Scan and Connect Wi-Fi

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
wifi scan
wifi connect MySSID MyPassword
wifi status
wifi wait 30
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Connect, wait, and verify IP address without rebooting.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

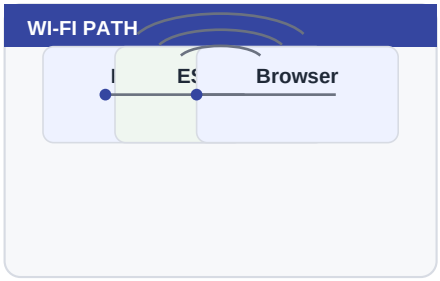
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

18. Save Wi-Fi Profiles

Profiles make it easy to move between networks or recover from credential changes.

Network features are a convenience layer over the same command engine. Use the serial console as the ground truth, then use the browser and API once the board is reachable.



The mental model

A profile is a named Wi-Fi memory.

Where you will use it

- Move the board from bench Wi-Fi to garden Wi-Fi.
- Find the IP address for the web UI.
- Recover access after a router or password change.

Terms to remember

- SSID: the Wi-Fi network name.
- DHCP: automatic IP address assignment by the router.
- Static IP: a manually chosen address for predictable access.

Why this matters

Installations in gardens and workshops often move during testing.

Common beginner mistake

Forgetting that saved credentials persist after reboot.

One-minute review

Explain save wi-fi profiles in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 18: Save Wi-Fi Profiles

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
wifi save HomeNetwork SecretPassword
wifi profile list
wifi autoconnect on
wifi reconnect
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Save a profile and inspect it before relying on it.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

19. Recover Wi-Fi

Recovery commands help when the board joins the wrong network or gets confused.

Recovery is part of design. If a command can start something, another command should let you inspect, pause or undo it.

The mental model

Wi-Fi recovery is a ladder: inspect, reconnect, forget, recover, SDK reset only when needed.

Where you will use it

- Pause automations before debugging.
- Remove a bad cron job without reflashing.
- Use serial as the rescue path when Wi-Fi is down.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

A calm recovery path prevents unnecessary reflashing.

Common beginner mistake

Using sdkreset as the first move instead of the last careful step.

One-minute review

Explain recover wi-fi in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 19: Recover Wi-Fi

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
wifi diag
wifi recover
wifi disconnect
wifi reconnect
wifi sdkreset --yes
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Write a recovery checklist before you need it.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

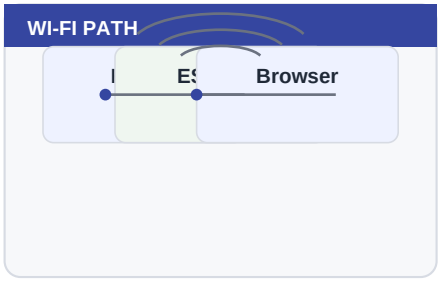
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

20. Static IP or DHCP

KernelESP can use DHCP or a static address depending on your installation.

Network features are a convenience layer over the same command engine. Use the serial console as the ground truth, then use the browser and API once the board is reachable.



The mental model

DHCP is convenient; static IP is predictable.

Where you will use it

- Move the board from bench Wi-Fi to garden Wi-Fi.
- Find the IP address for the web UI.
- Recover access after a router or password change.

Terms to remember

- SSID: the Wi-Fi network name.
- DHCP: automatic IP address assignment by the router.
- Static IP: a manually chosen address for predictable access.

Why this matters

Predictable addressing makes web access and API scripts easier.

Common beginner mistake

Setting a static IP outside the router subnet.

One-minute review

Explain static ip or dhcp in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 20: Static IP or DHCP

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
wifi dhcp on reconnect
wifi static 192.168.1.50 192.168.1.1 255.255.255.0 8.8.8.8 1.1.1.1 reconnect
wifi net
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Decide which mode is safer for your garden controller.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

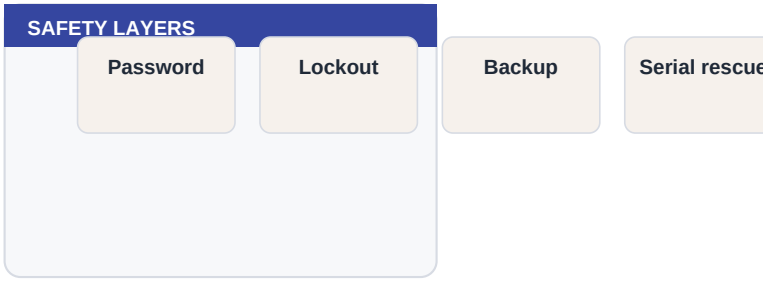
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

21. Fallback Access Point

The fallback AP can expose setup access when normal Wi-Fi fails.

Security on a microcontroller is mostly practical discipline: strong keys, private networks, no public exposure and a way to recover locally.



The mental model

AP mode is a temporary lifeboat, not the normal harbor.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Remote installations need a way back in after router changes.

Common beginner mistake

Leaving setup access open without thinking about physical security.

One-minute review

Explain fallback access point in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 21: Fallback Access Point

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
ap status
config get fallback.ap
config set fallback.ap on
ap start
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Confirm whether fallback AP is enabled on your board.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

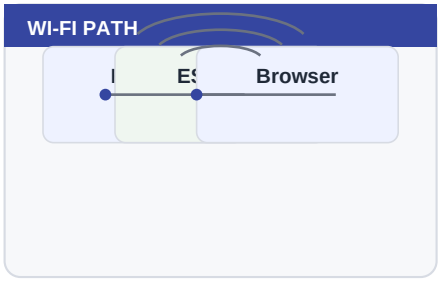
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

22. Set the Hostname

A good hostname makes the board easier to find and recognize.

Network features are a convenience layer over the same command engine. Use the serial console as the ground truth, then use the browser and API once the board is reachable.



The mental model

Names beat numbers when you have more than one controller.

Where you will use it

- Move the board from bench Wi-Fi to garden Wi-Fi.
- Find the IP address for the web UI.
- Recover access after a router or password change.

Terms to remember

- SSID: the Wi-Fi network name.
- DHCP: automatic IP address assignment by the router.
- Static IP: a manually chosen address for predictable access.

Why this matters

Clear names prevent editing the wrong controller.

Common beginner mistake

Using the same hostname for multiple boards.

One-minute review

Explain set the hostname in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 22: Set the Hostname

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
hostname
hostname kernelesp-garden
wifi reconnect
wifi status
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Choose a name that describes the physical location.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

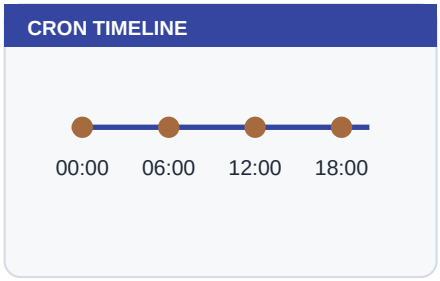
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

23. Read the Clock

Time unlocks cron, logs and safe irrigation windows.

Schedules are promises to the future. They should use 24-hour time, be easy to list, and have a safe way to pause them.



The mental model

The clock is the board's calendar. Without it, schedules cannot mean much.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Cron: clock-based scheduler.
- Timer: delay or interval-based scheduler.
- 24-hour time: 06:00, 12:00, 22:30.

Why this matters

Every schedule depends on trustworthy time.

Common beginner mistake

Assuming cron is wrong when NTP never synced.

One-minute review

Explain read the clock in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 23: Read the Clock

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
date
date -u
ntp status
time status
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Compare local time and UTC time.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

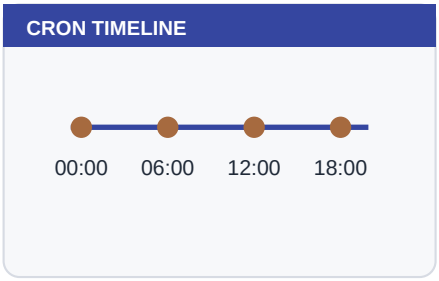
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

24. Use NTP Kick

ntp kick starts time synchronization without blocking the rest of the system.

Schedules are promises to the future. They should use 24-hour time, be easy to list, and have a safe way to pause them.



The mental model

It is a polite request: start syncing, then give the shell back.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Cron: clock-based scheduler.
- Timer: delay or interval-based scheduler.
- 24-hour time: 06:00, 12:00, 22:30.

Why this matters

Non-blocking time sync keeps automations responsive.

Common beginner mistake

Expecting instant perfect time on a weak Wi-Fi network.

One-minute review

Explain use ntp kick in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 24: Use NTP Kick

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
ntp kick
ntp status
date
dmesg | tail -n 5
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Run ntp kick, then keep using the shell while time settles.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

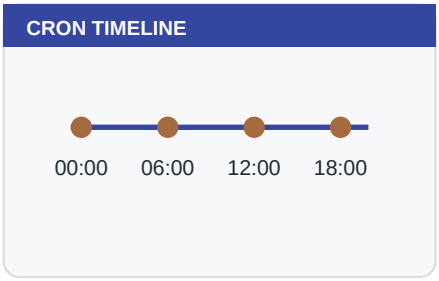
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

25. Schedule NTP Twice Daily

A simple cron pair keeps time fresh without hammering NTP servers.

Schedules are promises to the future. They should use 24-hour time, be easy to list, and have a safe way to pause them.



The mental model

Most garden controllers do not need minute-by-minute clock correction.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Cron: clock-based scheduler.
- Timer: delay or interval-based scheduler.
- 24-hour time: 06:00, 12:00, 22:30.

Why this matters

Twelve-hour sync is a practical balance for ESP8266 projects.

Common beginner mistake

Scheduling NTP every minute because more feels safer.

One-minute review

Explain schedule ntp twice daily in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 25: Schedule NTP Twice Daily

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
cron add daily 00:00 ntp kick
cron add daily 12:00 ntp kick
crontab -l
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Confirm both jobs appear in cron.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

26. Manual Time Rescue

Manual date setting is useful when Wi-Fi is down and you still need schedules.

Recovery is part of design. If a command can start something, another command should let you inspect, pause or undo it.

The mental model

Manual time is a temporary crutch until NTP returns.

Where you will use it

- Pause automations before debugging.
- Remove a bad cron job without reflashing.
- Use serial as the rescue path when Wi-Fi is down.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Time rescue can keep logs understandable during network outages.

Common beginner mistake

Forgetting to return to NTP after manual testing.

One-minute review

Explain manual time rescue in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 26: Manual Time Rescue

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
date
date set 2026-04-29 08:30:00
date
ntp kick
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Practice manual time on a bench board before field deployment.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

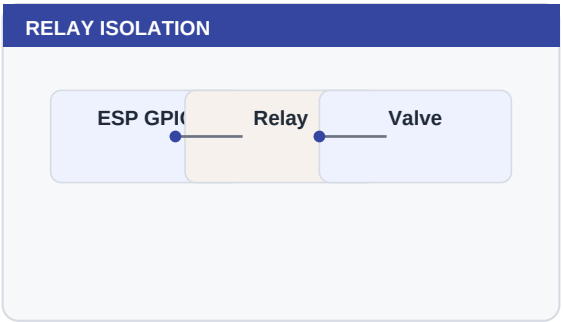
What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

PART 5

Hardware Control

This part covers Relay Basics, Name Relays Well, Pulse a Relay and builds toward practical confidence.

Chapters



How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

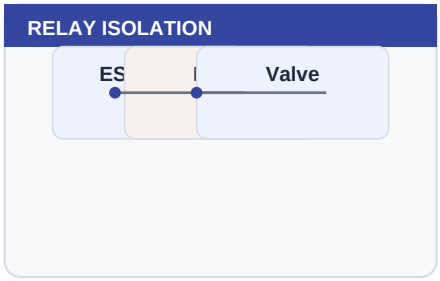
Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

27. Relay Basics

Relays let a 3.3 V logic pin control a separate load safely when wired correctly.

Hardware control is where software becomes physical. Treat every relay command as a real-world action, even while testing, and prefer names that describe the device being controlled.



The mental model

The ESP gives a signal; the relay handles the heavier electrical side.

Where you will use it

- Switch a fan, warning lamp, low-voltage valve or contactor input.
- Pulse a test load briefly before scheduling longer work.
- Keep relay names tied to physical labels on the enclosure.

Terms to remember

- Relay: an electrically controlled switch.
- Active-low: the relay turns on when the GPIO goes low.
- External supply: power source for relay coils or valves.

Why this matters

Relays are the bridge between software and physical action.

Common beginner mistake

Powering relay coils from the ESP 3.3 V pin.

One-minute review

Explain relay basics in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 27: Relay Basics

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay add valve1 D1 active_low
relay status
relay on valve1
relay off valve1
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Use dryrun until the relay module behavior is known.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

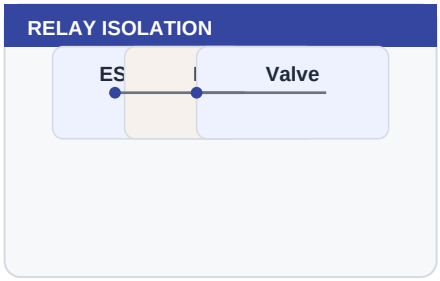
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

28. Name Relays Well

Good relay names make scripts readable and safer.

Hardware control is where software becomes physical. Treat every relay command as a real-world action, even while testing, and prefer names that describe the device being controlled.



The mental model

Names are labels for real-world equipment.

Where you will use it

- Switch a fan, warning lamp, low-voltage valve or contactor input.
- Pulse a test load briefly before scheduling longer work.
- Keep relay names tied to physical labels on the enclosure.

Terms to remember

- Relay: an electrically controlled switch.
- Active-low: the relay turns on when the GPIO goes low.
- External supply: power source for relay coils or valves.

Why this matters

Readable names reduce mistakes months later.

Common beginner mistake

Calling everything relay1, relay2 and forgetting which is which.

One-minute review

Explain name relays well in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 28: Name Relays Well

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay add trees D1 active_low
relay add flowers D2 active_low
relay status
relay rm flowers
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Choose names based on the thing controlled, not the GPIO number.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

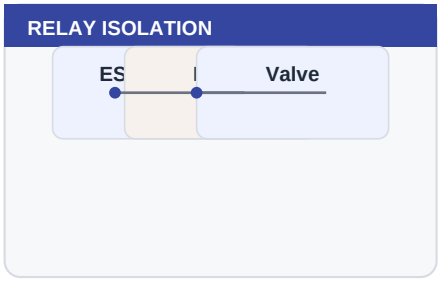
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

29. Pulse a Relay

A pulse turns something on briefly and then turns it off automatically.

Hardware control is where software becomes physical. Treat every relay command as a real-world action, even while testing, and prefer names that describe the device being controlled.



The mental model

Pulse is a short controlled action, useful for tests or momentary loads.

Where you will use it

- Switch a fan, warning lamp, low-voltage valve or contactor input.
- Pulse a test load briefly before scheduling longer work.
- Keep relay names tied to physical labels on the enclosure.

Terms to remember

- Relay: an electrically controlled switch.
- Active-low: the relay turns on when the GPIO goes low.
- External supply: power source for relay coils or valves.

Why this matters

Short actions are easier to validate than long schedules.

Common beginner mistake

Using pulse for long irrigation when a timer would be clearer.

One-minute review

Explain pulse a relay in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 29: Pulse a Relay

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay pulse valve1 500
relay status
log | tail -n 5
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Pulse a test LED before pulsing a real valve.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

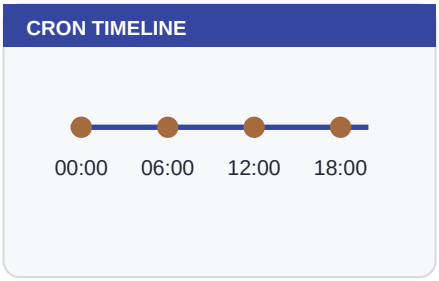
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

30. Timers Instead of Sleep

Timers schedule future actions without blocking the shell.

Schedules are promises to the future. They should use 24-hour time, be easy to list, and have a safe way to pause them.



The mental model

A timer is a reminder the system keeps while you do other things.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Cron: clock-based scheduler.
- Timer: delay or interval-based scheduler.
- 24-hour time: 06:00, 12:00, 22:30.

Why this matters

Non-blocking timers are essential for reliable automation.

Common beginner mistake

Using sleep for long actuator runs.

One-minute review

Explain timers instead of sleep in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 30: Timers Instead of Sleep

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay on valve1
timer once 600000 relay off valve1
timer list
relay status
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Start a short 10 second test timer and watch it complete.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

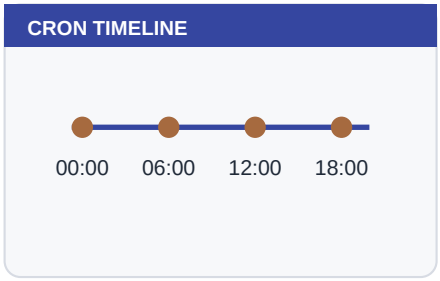
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

31. Repeating Timers

Repeating timers run commands at intervals while the system stays responsive.

Schedules are promises to the future. They should use 24-hour time, be easy to list, and have a safe way to pause them.



The mental model

They are useful for lightweight periodic maintenance.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Cron: clock-based scheduler.
- Timer: delay or interval-based scheduler.
- 24-hour time: 06:00, 12:00, 22:30.

Why this matters

Timers are simple when the interval matters more than the clock time.

Common beginner mistake

Leaving experimental timers running after tests.

One-minute review

Explain repeating timers in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 31: Repeating Timers

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
timer every 60000 health
timer list
timer clear
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Create a short repeating timer, observe it, then clear it.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

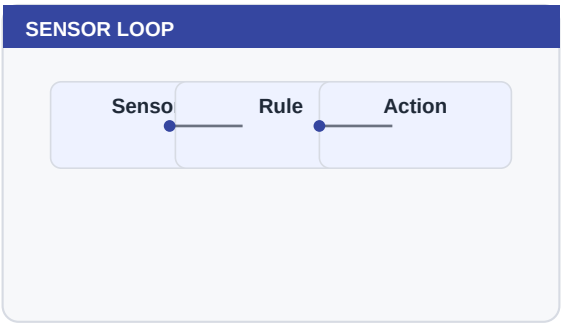
What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

PART 6

Scheduling and Sensors

This part covers Cron Daily Jobs, Cron Weekly Jobs, Cron and Safety and builds toward practical confidence.

Chapters



How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

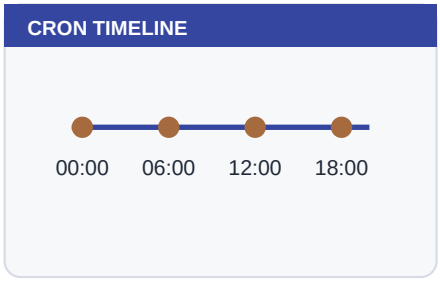
Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

32. Cron Daily Jobs

Cron runs commands at human clock times.

Schedules are promises to the future. They should use 24-hour time, be easy to list, and have a safe way to pause them.



The mental model

Cron is a calendar-based scheduler.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Cron: clock-based scheduler.
- Timer: delay or interval-based scheduler.
- 24-hour time: 06:00, 12:00, 22:30.

Why this matters

Daily cron jobs are perfect for irrigation windows.

Common beginner mistake

Adding on jobs without matching off or safety timers.

One-minute review

Explain cron daily jobs in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 32: Cron Daily Jobs

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
cron add daily 06:00 relay on trees
cron add daily 06:10 relay off trees
crontab -l
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Create a harmless logger job and remove it.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

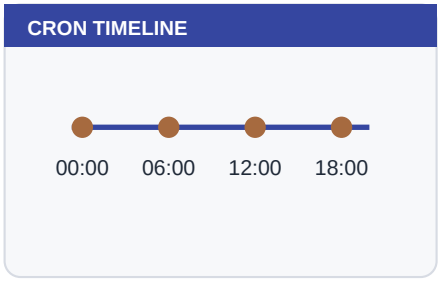
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

33. Cron Weekly Jobs

Weekly schedules support maintenance and weekly routines.

Schedules are promises to the future. They should use 24-hour time, be easy to list, and have a safe way to pause them.



The mental model

The clock decides when; the command engine decides what.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Cron: clock-based scheduler.
- Timer: delay or interval-based scheduler.
- 24-hour time: 06:00, 12:00, 22:30.

Why this matters

Weekly patterns keep maintenance visible.

Common beginner mistake

Forgetting local time and using AM/PM habits in a 24-hour system.

One-minute review

Explain cron weekly jobs in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 33: Cron Weekly Jobs

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
cron add weekly mon 08:00 logger monday_check
cron add weekly sun 22:30 sh /home/weekly.sh
crontab -l
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Schedule a weekly health log entry.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

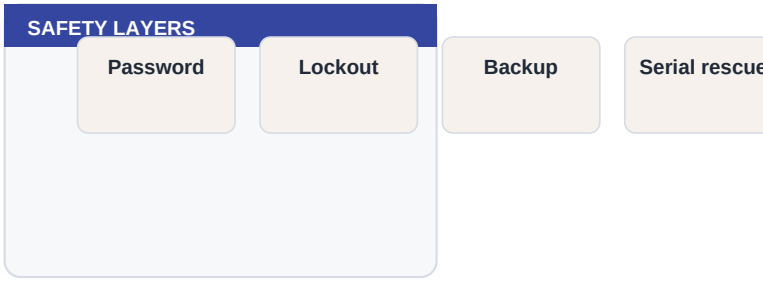
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

34. Cron and Safety

Cron jobs should be short, inspectable and reversible.

Security on a microcontroller is mostly practical discipline: strong keys, private networks, no public exposure and a way to recover locally.



The mental model

A schedule is only safe if the action is safe.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

A pause switch is a gift to your future self.

Common beginner mistake

Debugging a live schedule without disarming it first.

One-minute review

Explain cron and safety in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 34: Cron and Safety

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
crontab -l
cron rm <id>
armed
disarm
arm
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Find the quickest way to pause automations.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

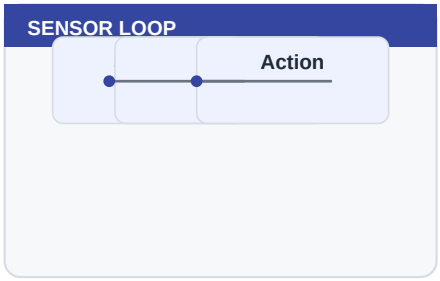
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

35. Sensors First Read

Sensors provide temperature, humidity and pressure values for decisions.

Sensors give the board facts about the environment. Good automation reads those facts, applies simple rules and logs the decision so a human can understand it later.



The mental model

Sensor values are measurements, not commands. Read before you react.

Where you will use it

- Turn on ventilation when temperature rises.
- Send an alert when pressure or humidity crosses a threshold.
- Use hysteresis so equipment does not chatter on and off.

Terms to remember

- I2C: a two-wire data bus used by many sensors.
- Threshold: the value where a rule changes behavior.
- Hysteresis: separate on and off thresholds to avoid chatter.

Why this matters

Rules and automations are only as good as the sensor data.

Common beginner mistake

Writing rules before confirming the sensor works.

One-minute review

Explain sensors first read in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 35: Sensors First Read

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
sensor begin
sensor read
sensor status
health
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Compare sensor output before and after touching the sensor gently.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

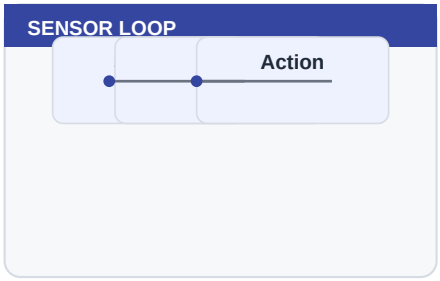
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

36. BME280 and BMP280

Environmental sensors can share I2C wiring when connected correctly.

Sensors give the board facts about the environment. Good automation reads those facts, applies simple rules and logs the decision so a human can understand it later.



The mental model

I2C is a small data bus: power, ground, clock and data.

Where you will use it

- Turn on ventilation when temperature rises.
- Send an alert when pressure or humidity crosses a threshold.
- Use hysteresis so equipment does not chatter on and off.

Terms to remember

- I2C: a two-wire data bus used by many sensors.
- Threshold: the value where a rule changes behavior.
- Hysteresis: separate on and off thresholds to avoid chatter.

Why this matters

Physical wiring errors often look like software bugs.

Common beginner mistake

Mixing 5 V sensor modules with 3.3 V-only GPIO without checking.

One-minute review

Explain bme280 and bmp280 in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 36: BME280 and BMP280

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
i2c scan
sensor begin
sensor read
dmesg | tail -n 10
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Run i2c scan and write down detected addresses.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

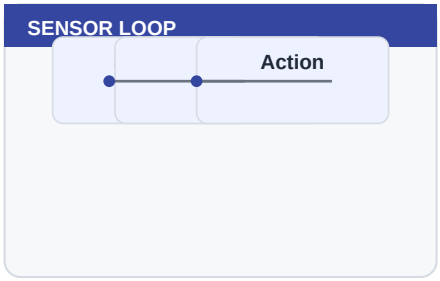
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

37. Sensor Rules

Rules turn measurements into actions.

Sensors give the board facts about the environment. Good automation reads those facts, applies simple rules and logs the decision so a human can understand it later.



The mental model

A rule is a small thermostat-like decision.

Where you will use it

- Turn on ventilation when temperature rises.
- Send an alert when pressure or humidity crosses a threshold.
- Use hysteresis so equipment does not chatter on and off.

Terms to remember

- I2C: a two-wire data bus used by many sensors.
- Threshold: the value where a rule changes behavior.
- Hysteresis: separate on and off thresholds to avoid chatter.

Why this matters

Rules are simpler than scripts for direct threshold behavior.

Common beginner mistake

Using one threshold for both on and off, causing rapid toggling.

One-minute review

Explain sensor rules in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 37: Sensor Rules

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
rule add temp gt 40 relay fan on
rule list
rule rm <id>
relay status
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Create a logger-only rule before controlling a relay.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

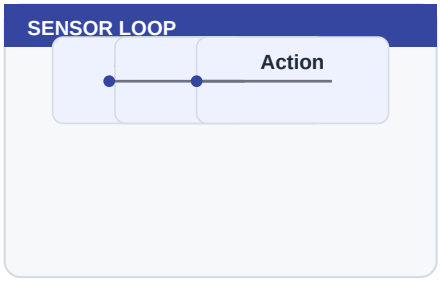
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

38. Hysteresis

Hysteresis prevents relays from chattering around a threshold.

Sensors give the board facts about the environment. Good automation reads those facts, applies simple rules and logs the decision so a human can understand it later.



The mental model

Use one threshold to turn on and another to turn off.

Where you will use it

- Turn on ventilation when temperature rises.
- Send an alert when pressure or humidity crosses a threshold.
- Use hysteresis so equipment does not chatter on and off.

Terms to remember

- I2C: a two-wire data bus used by many sensors.
- Threshold: the value where a rule changes behavior.
- Hysteresis: separate on and off thresholds to avoid chatter.

Why this matters

Mechanical relays and valves should not toggle every few seconds.

Common beginner mistake

Setting on at 40 and off at 39.9 in noisy conditions.

One-minute review

Explain hysteresis in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 38: Hysteresis

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
if (temp >= 40) relay on fan
if (temp <= 35) relay off fan
rule list
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Choose a gap wide enough for your real environment.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

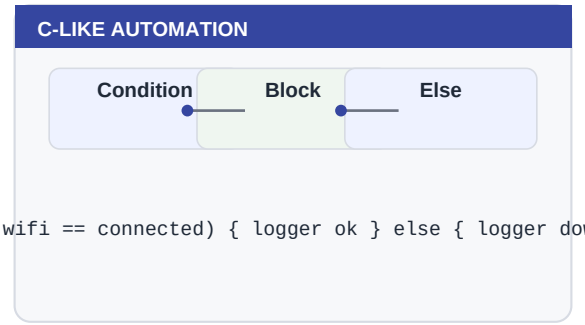
PART 7

Automation Language

This part covers Scenes, Persistent State, Digital Inputs and builds toward practical confidence.

Chapters

- Scenes
- Persistent State
- Digital Inputs
- Input Events
- C-like Expressions
- Blocks and Else
- Variables with let
- Constants with define
- Functions
- Scripts
- Boot Script



How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

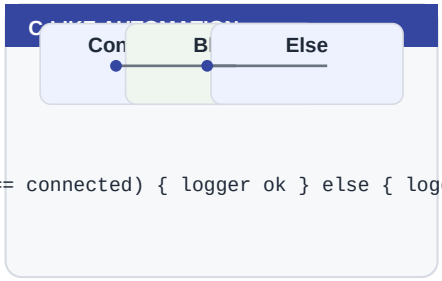
Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

39. Scenes

Scenes group commands into named actions.

Automation is just a repeatable decision. The best programs stay short, use clear names and can be run manually before they are scheduled.




```
if (wifi == connected) { logger ok } else { logger down }
```

The mental model

A scene is a macro for a state you want often.

Where you will use it

- Express rules such as if temperature is high and Wi-Fi is connected, send an alert.
- Store reusable actions as functions and call them from cron.
- Use persistent variables as user-friendly switches such as irrigation.enabled.

Terms to remember

- Expression: a yes/no condition.
- Block: several commands grouped with braces.
- Function: a persistent named command block.

Why this matters

Scenes make web buttons and scripts easier to understand.

Common beginner mistake

Putting too much logic inside a scene; use scripts for logic.

One-minute review

Explain scenes in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 39: Scenes

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
scene add night relay off lights; logger night_scene
scene run night
scene list
scene show night
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Create a scene that only logs, then inspect it.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

40. Persistent State

State stores small values that survive reboot.

Files make the board persistent. A command changes the current moment; a file lets the board remember a script, configuration or note after reboot.

LITTLEFS LAYOUT

```
/
  /etc      config, cron, relays
  /home     scripts and notes
  /www      web interface
  /var/log  kernel.log
  /func     persistent functions
```

The mental model

State is the board's notebook.

Where you will use it

- Store scripts, installation notes and boot commands.
- Inspect configuration without recompiling firmware.
- Back up the board before experiments.

Terms to remember

- LittleFS: the small flash filesystem used by KernelESP.
- /etc: persistent configuration.
- /home: user scripts and notes.

Why this matters

Persistent state lets automations remember modes.

Common beginner mistake

Using state for large data logs instead of small key values.

One-minute review

Explain persistent state in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 40: Persistent State

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
state set irrigation.enabled 1
state get irrigation.enabled
state list
state rm irrigation.enabled
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Store a harmless value and confirm it persists after reboot later.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

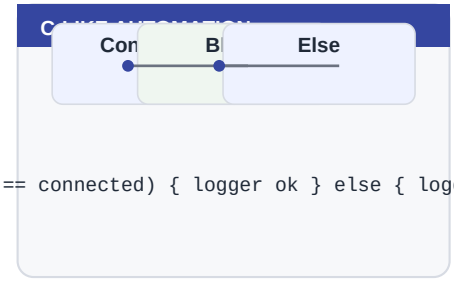
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

41. Digital Inputs

Inputs react to buttons, switches and sensor pulses.

Automation is just a repeatable decision. The best programs stay short, use clear names and can be run manually before they are scheduled.




```
if (wifi == connected) { logger ok } else { logger down }
```

The mental model

An input is the board listening to a pin.

Where you will use it

- Express rules such as if temperature is high and Wi-Fi is connected, send an alert.
- Store reusable actions as functions and call them from cron.
- Use persistent variables as user-friendly switches such as irrigation.enabled.

Terms to remember

- Expression: a yes/no condition.
- Block: several commands grouped with braces.
- Function: a persistent named command block.

Why this matters

Names make input automations readable.

Common beginner mistake

Forgetting pullup behavior and reading inverted logic.

One-minute review

Explain digital inputs in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 41: Digital Inputs

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
input add rain D2 pullup 50
input list
input rm rain
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Add a named input even before wiring the final sensor.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

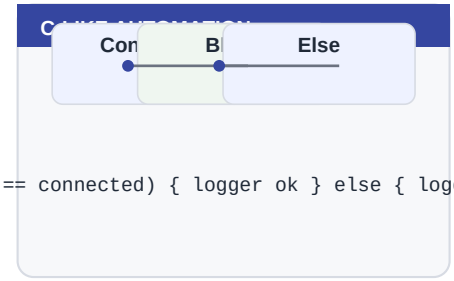
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

42. Input Events

Input events attach commands to high, low or change transitions.

Automation is just a repeatable decision. The best programs stay short, use clear names and can be run manually before they are scheduled.




```
if (wifi == connected) { logger ok } else { logger down }
```

The mental model

The event is the trigger; the command is the response.

Where you will use it

- Express rules such as if temperature is high and Wi-Fi is connected, send an alert.
- Store reusable actions as functions and call them from cron.
- Use persistent variables as user-friendly switches such as irrigation.enabled.

Terms to remember

- Expression: a yes/no condition.
- Block: several commands grouped with braces.
- Function: a persistent named command block.

Why this matters

Logging input events proves your wiring before actuation.

Common beginner mistake

Starting a pump from an untested bouncing switch.

One-minute review

Explain input events in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 42: Input Events

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
input add button D7 pullup 50
input on button low logger button_pressed
input list
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Use logger first, hardware second.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

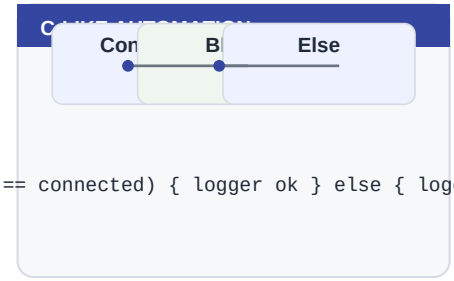
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

43. C-like Expressions

The automation language supports familiar comparisons and logic.

Automation is just a repeatable decision. The best programs stay short, use clear names and can be run manually before they are scheduled.




```
if (wifi == connected) { logger ok } else { logger down }
```

The mental model

Expressions answer yes or no before running a command.

Where you will use it

- Express rules such as if temperature is high and Wi-Fi is connected, send an alert.
- Store reusable actions as functions and call them from cron.
- Use persistent variables as user-friendly switches such as irrigation.enabled.

Terms to remember

- Expression: a yes/no condition.
- Block: several commands grouped with braces.
- Function: a persistent named command block.

Why this matters

Readable conditions make complex projects approachable.

Common beginner mistake

Expecting full C. This is a tiny expression layer, not a compiler.

One-minute review

Explain c-like expressions in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 43: C-like Expressions

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
if (wifi == connected) logger online
if (armed == on && wifi == connected) logger ready
if (!(armed == on)) logger paused
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Change armed state and observe which condition fires.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

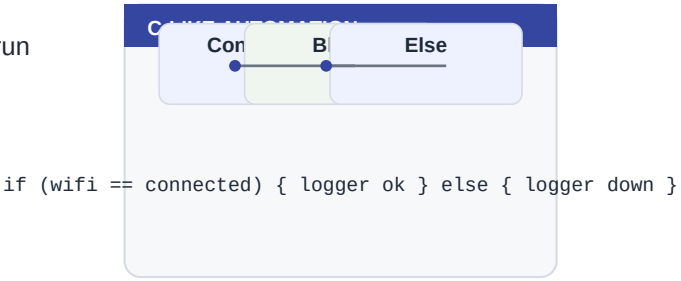
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

44. Blocks and Else

Blocks let one condition run several commands.

Automation is just a repeatable decision. The best programs stay short, use clear names and can be run manually before they are scheduled.



The mental model

A block is a small group of commands separated by semicolons.

Where you will use it

- Express rules such as if temperature is high and Wi-Fi is connected, send an alert.
- Store reusable actions as functions and call them from cron.
- Use persistent variables as user-friendly switches such as irrigation.enabled.

Terms to remember

- Expression: a yes/no condition.
- Block: several commands grouped with braces.
- Function: a persistent named command block.

Why this matters

Blocks reduce duplicated conditions.

Common beginner mistake

Writing multiline if blocks in scripts; KernelESP rejects them for stability.

One-minute review

Explain blocks and else in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 44: Blocks and Else

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
if (wifi == connected) { logger online; mail health "online" } else { logger offline }
if (armed == on) { relay status; timer list }
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Keep blocks on one line in scripts.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

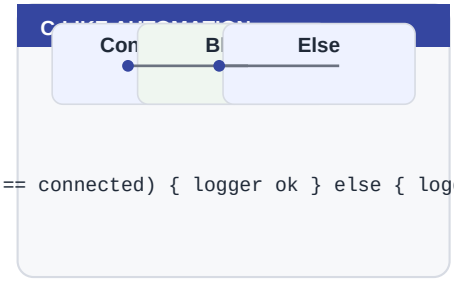
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

45. Variables with let

let creates persistent variables for automation decisions.

Automation is just a repeatable decision. The best programs stay short, use clear names and can be run manually before they are scheduled.




```
if (wifi == connected) { logger ok } else { logger down }
```

The mental model

Variables are readable knobs for behavior.

Where you will use it

- Express rules such as if temperature is high and Wi-Fi is connected, send an alert.
- Store reusable actions as functions and call them from cron.
- Use persistent variables as user-friendly switches such as irrigation.enabled.

Terms to remember

- Expression: a yes/no condition.
- Block: several commands grouped with braces.
- Function: a persistent named command block.

Why this matters

Variables let beginners tune behavior without rewriting every command.

Common beginner mistake

Forgetting that variable names are case-insensitive.

One-minute review

Explain variables with let in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 45: Variables with let

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
let irrigation.enabled = 1
let max.temp = 35
let list
if (irrigation.enabled == 1 && temp < max.temp) logger allowed
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Create a variable for your preferred watering duration.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

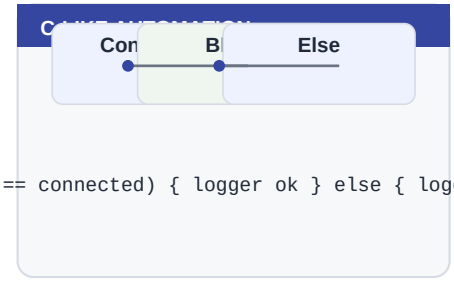
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

46. Constants with define

define creates named constants used in expressions.

Automation is just a repeatable decision. The best programs stay short, use clear names and can be run manually before they are scheduled.




```
if (wifi == connected) { logger ok } else { logger down }
```

The mental model

Constants are labels for numbers and times.

Where you will use it

- Express rules such as if temperature is high and Wi-Fi is connected, send an alert.
- Store reusable actions as functions and call them from cron.
- Use persistent variables as user-friendly switches such as irrigation.enabled.

Terms to remember

- Expression: a yes/no condition.
- Block: several commands grouped with braces.
- Function: a persistent named command block.

Why this matters

Names explain intent better than bare numbers.

Common beginner mistake

Assuming HOT and hot are different constants.

One-minute review

Explain constants with define in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 46: Constants with define

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
define HOT 40
define MORNING_END 10:00
define list
if (temp >= HOT && time < MORNING_END) logger hot_morning
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Replace a magic number in one script with a constant.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

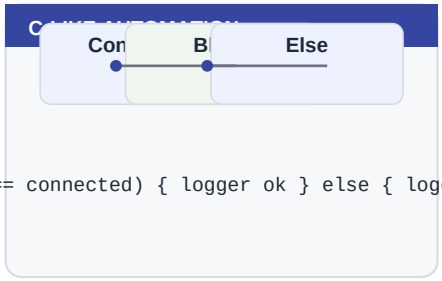
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

47. Functions

Functions store named command blocks under /func.

Automation is just a repeatable decision. The best programs stay short, use clear names and can be run manually before they are scheduled.




```
if (wifi == connected) { logger ok } else { logger down }
```

The mental model

A function is a reusable mini script.

Where you will use it

- Express rules such as if temperature is high and Wi-Fi is connected, send an alert.
- Store reusable actions as functions and call them from cron.
- Use persistent variables as user-friendly switches such as irrigation.enabled.

Terms to remember

- Expression: a yes/no condition.
- Block: several commands grouped with braces.
- Function: a persistent named command block.

Why this matters

Functions keep cron and input actions short.

Common beginner mistake

Calling functions from inside if branches; use direct commands in branches.

One-minute review

Explain functions in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 47: Functions

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
function water_zone1 { relay on valve1; timer once 600000 relay off valve1; logger zone1_started
function list
function show water_zone1
call water_zone1
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Create a function that only logs before using relays.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

48. Scripts

Scripts are files containing commands, one per line.

Files make the board persistent. A command changes the current moment; a file lets the board remember a script, configuration or note after reboot.

LITTLEFS LAYOUT

```
/
  /etc      config, cron, relays
  /home     scripts and notes
  /www      web interface
  /var/log  kernel.log
  /func     persistent functions
```

The mental model

A script is a recipe stored on the board.

Where you will use it

- Store scripts, installation notes and boot commands.
- Inspect configuration without recompiling firmware.
- Back up the board before experiments.

Terms to remember

- LittleFS: the small flash filesystem used by KernelESP.
- /etc: persistent configuration.
- /home: user scripts and notes.

Why this matters

Scripts make setup repeatable.

Common beginner mistake

Saving untested scripts into boot.

One-minute review

Explain scripts in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 48: Scripts

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
write /home/test.sh echo start
append /home/test.sh health
sh -n /home/test.sh
sh /home/test.sh
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Validate scripts with sh -n before running them.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

49. Boot Script

The boot script runs startup commands.

Recovery is part of design. If a command can start something, another command should let you inspect, pause or undo it.

The mental model

Boot is the board's morning routine.

Where you will use it

- Pause automations before debugging.
- Remove a bad cron job without reflashing.
- Use serial as the rescue path when Wi-Fi is down.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

A safe boot script makes power loss recovery boring.

Common beginner mistake

Putting long sleeps or risky relay actions at boot.

One-minute review

Explain boot script in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 49: Boot Script

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
boot show
boot set /etc/boot.sh
cat /etc/boot.sh
onboot list
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Keep boot commands short and non-blocking.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

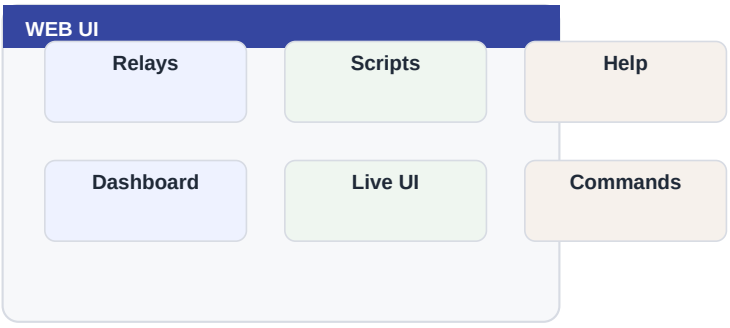
PART 8

Web, Mail and Diagnostics

This part covers Logs, Backups, Mail Setup and builds toward practical confidence.

Chapters

- Logs
- Backups
- Mail Setup
- Mail Alerts
- Diagnostics
- Live UI
- Command Runner
- Script Editor
- Local Help
- Security Basics
- Web Lockout



How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

50. Logs

Logs tell the story of what happened.

Files make the board persistent. A command changes the current moment; a file lets the board remember a script, configuration or note after reboot.

LITTLEFS LAYOUT

```
/
  /etc      config, cron, relays
  /home     scripts and notes
  /www      web interface
  /var/log  kernel.log
  /func     persistent functions
```

The mental model

When confused, read the diary.

Where you will use it

- Store scripts, installation notes and boot commands.
- Inspect configuration without recompiling firmware.
- Back up the board before experiments.

Terms to remember

- LittleFS: the small flash filesystem used by KernelESP.
- /etc: persistent configuration.
- /home: user scripts and notes.

Why this matters

Logs turn mystery failures into timelines.

Common beginner mistake

Clearing logs before copying the useful clue.

One-minute review

Explain logs in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 50: Logs

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
log
tail -n 20 /var/log/kernel.log
grep cron /var/log/kernel.log
dmesg
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Find the last Wi-Fi-related log entry.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

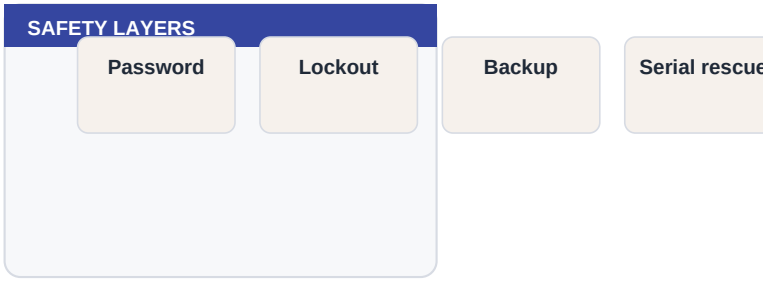
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

51. Backups

Backups preserve configuration before experiments.

Security on a microcontroller is mostly practical discipline: strong keys, private networks, no public exposure and a way to recover locally.



The mental model

A backup is a snapshot you can return to.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Small boards are easier to experiment with when rollback exists.

Common beginner mistake

Testing risky automation without a current backup.

One-minute review

Explain backups in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 51: Backups

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
backup
export relays
cat /etc/config.txt
ls /etc
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Make a backup before changing Wi-Fi or cron.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

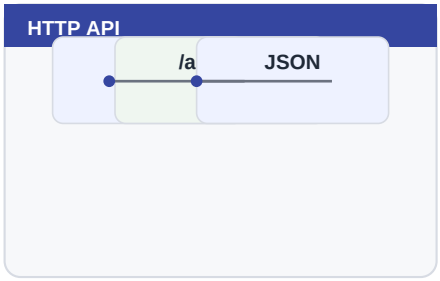
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

52. Mail Setup

Mail alerts let the board report important events.

The API is a bridge to other computers. Start with read-only status calls, then add control only when local safety and logging are already in place.



The mental model

Email is an outbound notification path.

Where you will use it

- Read status from a laptop or home dashboard.
- Trigger a command from another local automation system.
- Build tests that verify the board is alive after updates.

Terms to remember

- Endpoint: a URL that performs a specific task.
- JSON: structured text returned by the API.
- URL encoding: safe representation of spaces and symbols.

Why this matters

Notifications are most useful when they are boringly reliable.

Common beginner mistake

Triggering emails every few seconds from a noisy rule.

One-minute review

Explain mail setup in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 52: Mail Setup

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
mail status
mail config
mail test
mail health "KernelESP health"
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Send a test email before using mail in automation.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

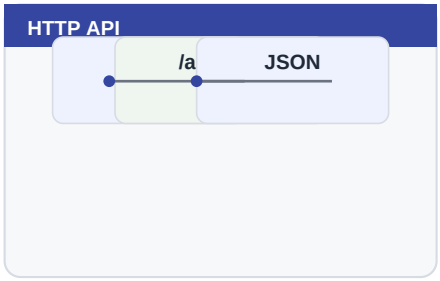
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

53. Mail Alerts

A good alert says what happened and what to do next.

The API is a bridge to other computers. Start with read-only status calls, then add control only when local safety and logging are already in place.



The mental model

Alerts are for humans, not logs with wings.

Where you will use it

- Read status from a laptop or home dashboard.
- Trigger a command from another local automation system.
- Build tests that verify the board is alive after updates.

Terms to remember

- Endpoint: a URL that performs a specific task.
- JSON: structured text returned by the API.
- URL encoding: safe representation of spaces and symbols.

Why this matters

Useful alerts make remote installs manageable.

Common beginner mistake

Sending alert storms during unstable Wi-Fi.

One-minute review

Explain mail alerts in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 53: Mail Alerts

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
if (wifi == connected) mail health "daily health"
logger daily_health_sent
mail status
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Create one daily health email, not ten noisy alerts.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

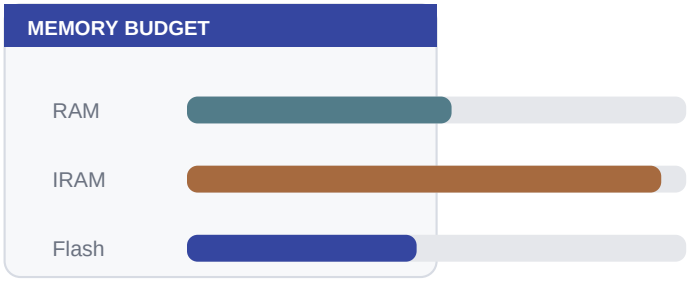
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

54. Diagnostics

Diagnostics bundle the facts needed for support.

Memory is a budget. KernelESP can do a lot because it keeps features small and avoids long blocking work.



The mental model

A diagnostic is a snapshot of the system state.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Good diagnostics reduce guesswork.

Common beginner mistake

Only reporting symptoms without command output.

One-minute review

Explain diagnostics in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 54: Diagnostics

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
diag
board
health
sysinfo
wifi diag
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Run diag before and after a major change.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

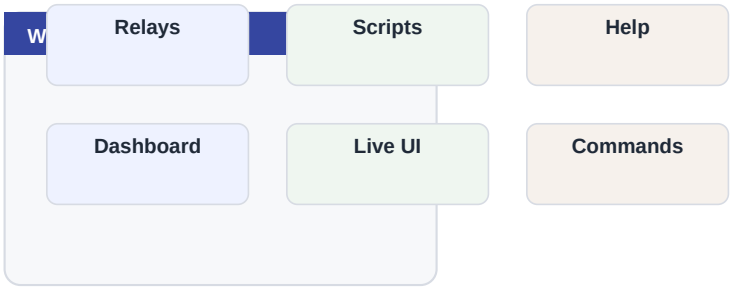
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

55. Live UI

Live UI shows changing system state in the browser.

The web UI is a friendly window into KernelESP. It should make common work easier, but the underlying commands remain the source of truth.



The mental model

It is a dashboard, not a replacement for careful command checks.

Where you will use it

- Run commands without opening a serial terminal.
- Edit scripts from a browser.
- Read local help when the internet is unavailable.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Refreshing less often keeps the ESP responsive.

Common beginner mistake

Leaving many browser tabs polling the board during critical automation.

One-minute review

Explain live ui in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 55: Live UI

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
health
free
timer list
crontab -l
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Open Live UI and watch heap while running harmless commands.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

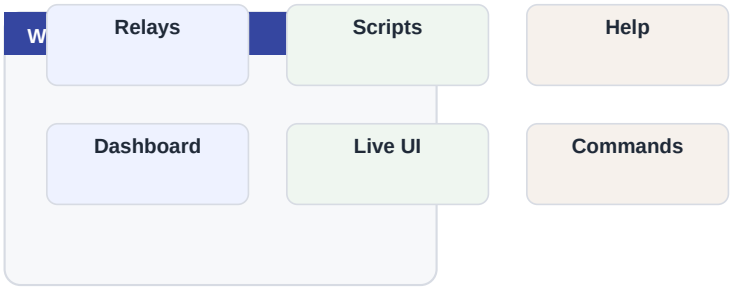
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

56. Command Runner

The web command runner is a beginner-friendly shell.

The web UI is a friendly window into KernelESP. It should make common work easier, but the underlying commands remain the source of truth.



The mental model

It is ideal for controlled experiments from a browser.

Where you will use it

- Run commands without opening a serial terminal.
- Edit scripts from a browser.
- Read local help when the internet is unavailable.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Seeing identical behavior builds trust in the model.

Common beginner mistake

Pasting a long unreviewed command from notes.

One-minute review

Explain command runner in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 56: Command Runner

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
echo hello
free
relay status
help relay
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Run the same command in serial and web and compare outputs.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

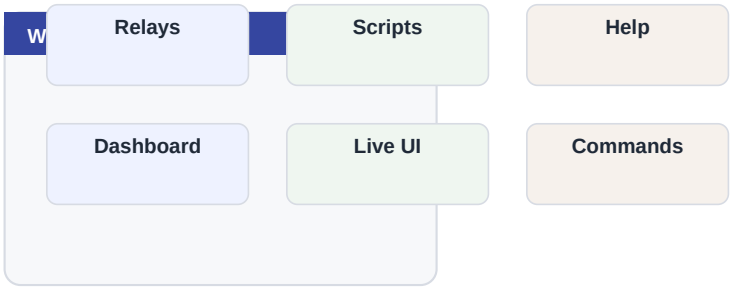
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

57. Script Editor

The web editor makes scripts accessible without a serial terminal.

The web UI is a friendly window into KernelESP. It should make common work easier, but the underlying commands remain the source of truth.



The mental model

It is a small file editor for /home and /etc scripts.

Where you will use it

- Run commands without opening a serial terminal.
- Edit scripts from a browser.
- Read local help when the internet is unavailable.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Validation catches simple mistakes before automation does.

Common beginner mistake

Editing boot scripts directly without backup.

One-minute review

Explain script editor in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 57: Script Editor

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
ls /home
cat /home/test.sh
sh -n /home/test.sh
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Edit a script, validate, save, then run.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

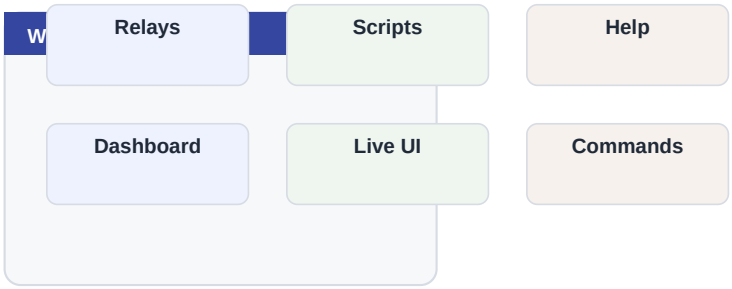
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

58. Local Help

Help files live on the board so documentation is available offline.

The web UI is a friendly window into KernelESP. It should make common work easier, but the underlying commands remain the source of truth.



The mental model

The board carries its own pocket manual.

Where you will use it

- Run commands without opening a serial terminal.
- Edit scripts from a browser.
- Read local help when the internet is unavailable.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Offline help is valuable in gardens, workshops and basements.

Common beginner mistake

Assuming online docs are required for every small task.

One-minute review

Explain local help in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 58: Local Help

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
help
help relay
ls /help
cat /help/scripts.txt
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Open the Help page in the web UI while disconnected from the internet.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

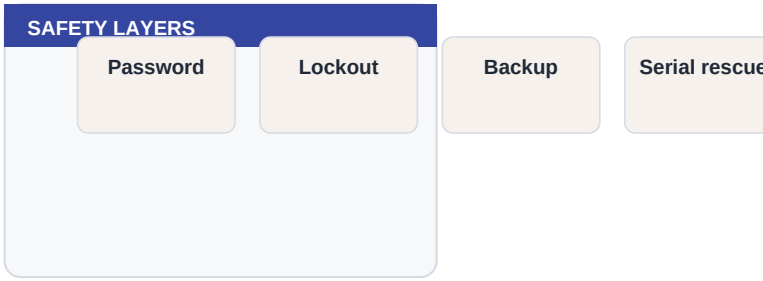
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

59. Security Basics

Security starts with a good web key and careful network exposure.

Security on a microcontroller is mostly practical discipline: strong keys, private networks, no public exposure and a way to recover locally.



The mental model

The board should be reachable by you, not by the whole world.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Garden automation can affect real hardware, so access matters.

Common beginner mistake

Port-forwarding the ESP web UI to the internet.

One-minute review

Explain security basics in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 59: Security Basics

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
config get web.key
config set web.key <new-key>
ap status
wifi net
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Change the web key on a private bench network first.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

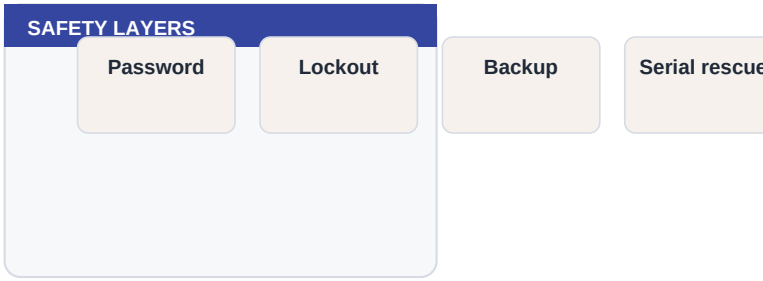
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

60. Web Lockout

Lockout slows repeated bad password attempts.

Security on a microcontroller is mostly practical discipline: strong keys, private networks, no public exposure and a way to recover locally.



The mental model

It is a seatbelt, not a castle wall.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Even small devices benefit from friction against guessing.

Common beginner mistake

Depending on lockout while using a weak password.

One-minute review

Explain web lockout in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 60: Web Lockout

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
config get web.lockout
health
diag
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Confirm lockout settings before field installation.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

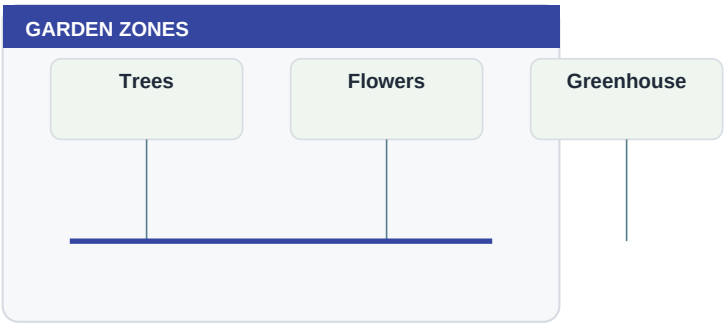
What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

PART 9

Garden Projects

This part covers GPIO Choices, External Relay Power, Long Cable Runs and builds toward practical confidence.

Chapters



How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

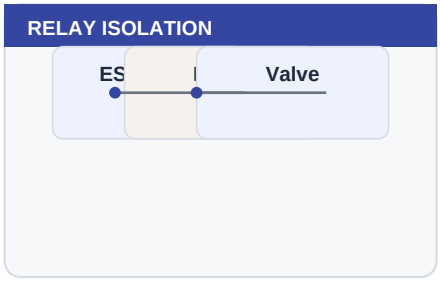
Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

61. GPIO Choices

Not every GPIO is equally safe for relays and sensors.

Hardware control is where software becomes physical. Treat every relay command as a real-world action, even while testing, and prefer names that describe the device being controlled.



The mental model

Some pins affect boot mode; choose conservative pins first.

Where you will use it

- Switch a fan, warning lamp, low-voltage valve or contactor input.
- Pulse a test load briefly before scheduling longer work.
- Keep relay names tied to physical labels on the enclosure.

Terms to remember

- Relay: an electrically controlled switch.
- Active-low: the relay turns on when the GPIO goes low.
- External supply: power source for relay coils or valves.

Why this matters

Correct pin choice prevents boot surprises.

Common beginner mistake

Using boot-sensitive pins without understanding pullups.

One-minute review

Explain gpio choices in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 61: GPIO Choices

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
board pins
board show
pin D1
pin D2
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Map every wire to a named relay or input.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

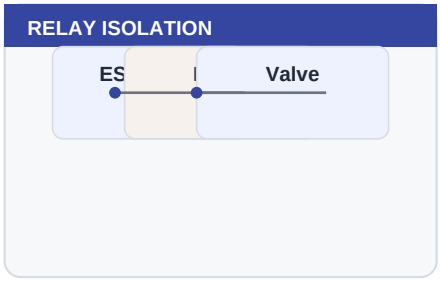
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

62. External Relay Power

Relay modules should usually have their own suitable power supply.

Hardware control is where software becomes physical. Treat every relay command as a real-world action, even while testing, and prefer names that describe the device being controlled.



The mental model

The ESP sends logic; the external supply does the coil work.

Where you will use it

- Switch a fan, warning lamp, low-voltage valve or contactor input.
- Pulse a test load briefly before scheduling longer work.
- Keep relay names tied to physical labels on the enclosure.

Terms to remember

- Relay: an electrically controlled switch.
- Active-low: the relay turns on when the GPIO goes low.
- External supply: power source for relay coils or valves.

Why this matters

Stable power is more important than elegant code.

Common beginner mistake

Powering many relay coils from the ESP regulator.

One-minute review

Explain external relay power in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 62: External Relay Power

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay status
free
health
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Measure relay supply voltage while toggling.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

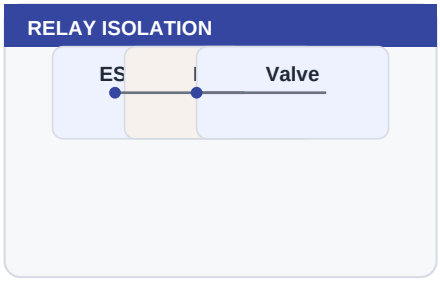
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

63. Long Cable Runs

Long cables need planning for voltage drop, noise and protection.

Hardware control is where software becomes physical. Treat every relay command as a real-world action, even while testing, and prefer names that describe the device being controlled.



The mental model

A cable is part of the circuit, not just a line on a drawing.

Where you will use it

- Switch a fan, warning lamp, low-voltage valve or contactor input.
- Pulse a test load briefly before scheduling longer work.
- Keep relay names tied to physical labels on the enclosure.

Terms to remember

- Relay: an electrically controlled switch.
- Active-low: the relay turns on when the GPIO goes low.
- External supply: power source for relay coils or valves.

Why this matters

Field wiring fails differently from bench wiring.

Common beginner mistake

Assuming a 100 m cable behaves like a 10 cm jumper.

One-minute review

Explain long cable runs in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 63: Long Cable Runs

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
logger cable_test_start
relay pulse valve1 500
logger cable_test_done
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Test with the actual cable length before burying anything.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

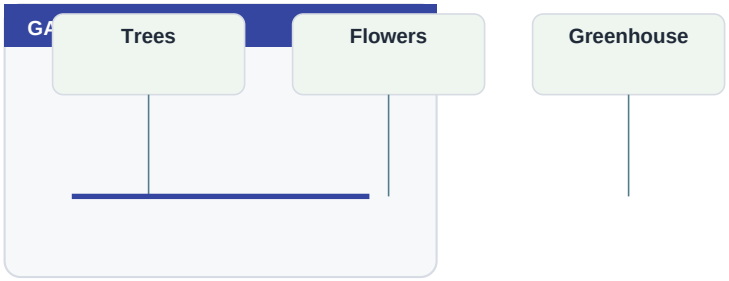
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

64. Outdoor Enclosures

Outdoor electronics need protection from water, condensation, sun and frost.

Garden automation combines water, electricity, weather and time. That makes it useful, but also worth testing carefully with one zone before expanding.



The mental model

An enclosure is climate control for a tiny computer.

Where you will use it

- Control tree, flower or greenhouse watering zones with solenoid valves.
- Skip watering when rain, heat or time windows make irrigation unsafe.
- Log every watering decision so you can troubleshoot dry plants or wasted water.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Mechanical protection is reliability work.

Common beginner mistake

Putting a bare relay board in direct sun or frost.

One-minute review

Explain outdoor enclosures in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 64: Outdoor Enclosures

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
health
sensor read
log | tail -n 10
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Design for drip loops, strain relief and service access.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

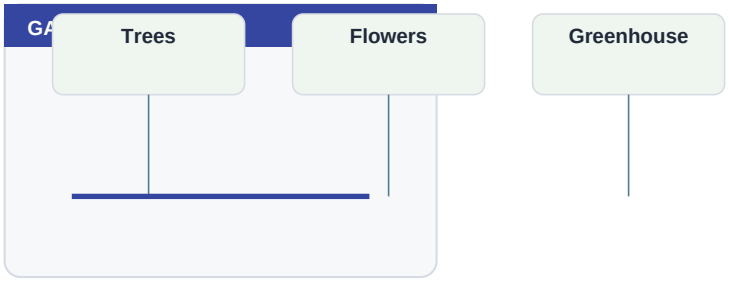
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

65. Solenoid Valves

Irrigation valves are usually better described as solenoids than pumps.

Garden automation combines water, electricity, weather and time. That makes it useful, but also worth testing carefully with one zone before expanding.



The mental model

The relay energizes a valve; water pressure does the work.

Where you will use it

- Control tree, flower or greenhouse watering zones with solenoid valves.
- Skip watering when rain, heat or time windows make irrigation unsafe.
- Log every watering decision so you can troubleshoot dry plants or wasted water.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Solenoids make zone control practical.

Common beginner mistake

Mixing AC and DC valve assumptions.

One-minute review

Explain solenoid valves in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 65: Solenoid Valves

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay add trees D1 active_low
relay on trees
timer once 900000 relay off trees
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Choose valve voltage before choosing the power supply.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

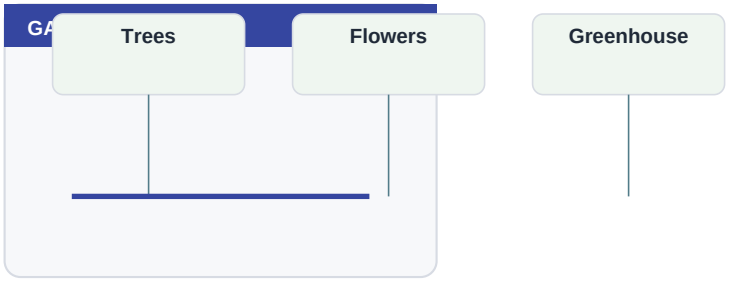
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

66. Flow Measurement

A flow sensor can confirm that watering actually happened.

Garden automation combines water, electricity, weather and time. That makes it useful, but also worth testing carefully with one zone before expanding.



The mental model

Relay on means command sent; flow means water moved.

Where you will use it

- Control tree, flower or greenhouse watering zones with solenoid valves.
- Skip watering when rain, heat or time windows make irrigation unsafe.
- Log every watering decision so you can troubleshoot dry plants or wasted water.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Feedback catches closed taps, clogged filters and broken wires.

Common beginner mistake

Trusting relay state as proof of water delivery.

One-minute review

Explain flow measurement in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 66: Flow Measurement

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
input add flow D7 pullup 20
input on flow change logger flow_pulse
log | tail -n 20
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Log pulses before converting them into liters.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

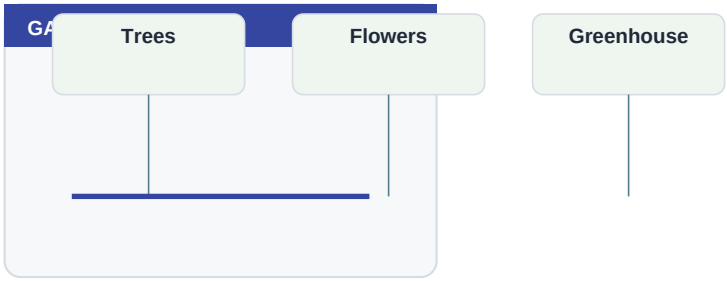
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

67. Build Zone One

A first irrigation zone should be simple, observable and reversible.

Garden automation combines water, electricity, weather and time. That makes it useful, but also worth testing carefully with one zone before expanding.



The mental model

One zone is your learning sandbox.

Where you will use it

- Control tree, flower or greenhouse watering zones with solenoid valves.
- Skip watering when rain, heat or time windows make irrigation unsafe.
- Log every watering decision so you can troubleshoot dry plants or wasted water.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

A small first success makes the bigger system safer.

Common beginner mistake

Building four zones before proving one zone.

One-minute review

Explain build zone one in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 67: Build Zone One

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay add trees D1 active_low
function water_trees { relay on trees; timer once 900000 relay off trees; logger trees_started
call water_trees
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Run the function with water off first.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

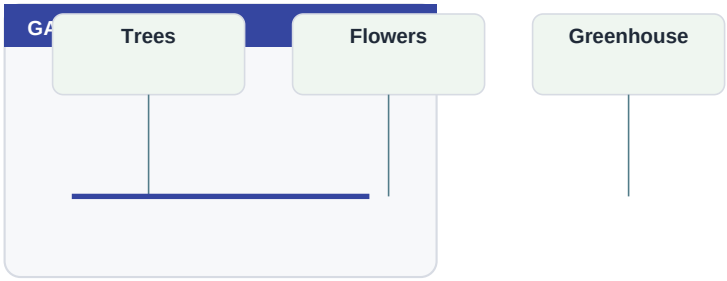
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

68. Add Zone Two

Second zones teach naming, schedule spacing and power budgeting.

Garden automation combines water, electricity, weather and time. That makes it useful, but also worth testing carefully with one zone before expanding.



The mental model

Zones should not fight for water pressure or relay power.

Where you will use it

- Control tree, flower or greenhouse watering zones with solenoid valves.
- Skip watering when rain, heat or time windows make irrigation unsafe.
- Log every watering decision so you can troubleshoot dry plants or wasted water.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Spacing zones is kinder to plumbing and power supplies.

Common beginner mistake

Starting every valve at the same minute.

One-minute review

Explain add zone two in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 68: Add Zone Two

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
relay add flowers D2 active_low
function water_flowers { relay on flowers; timer once 300000 relay off flowers; logger flowers_
cron add daily 06:20 call water_flowers
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Leave time between zones and observe pressure.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

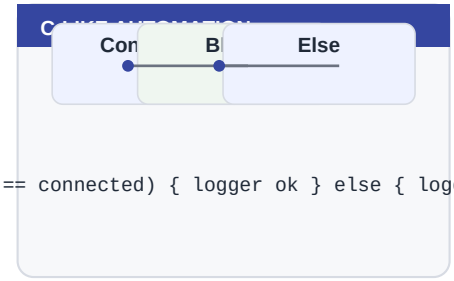
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

69. Rain Skip Pattern

A rain input can pause watering.

Automation is just a repeatable decision. The best programs stay short, use clear names and can be run manually before they are scheduled.




```
if (wifi == connected) { logger ok } else { logger down }
```

The mental model

A sensor changes a variable; schedules read that variable.

Where you will use it

- Express rules such as if temperature is high and Wi-Fi is connected, send an alert.
- Store reusable actions as functions and call them from cron.
- Use persistent variables as user-friendly switches such as irrigation.enabled.

Terms to remember

- Expression: a yes/no condition.
- Block: several commands grouped with braces.
- Function: a persistent named command block.

Why this matters

Separating sensing from watering keeps logic understandable.

Common beginner mistake

Letting a noisy sensor directly toggle a valve.

One-minute review

Explain rain skip pattern in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 69: Rain Skip Pattern

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
let irrigation.enabled = 1
input add rain D5 pullup 50
when input rain low let irrigation.enabled = 0
let list
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Use logger first if your rain sensor logic is unknown.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

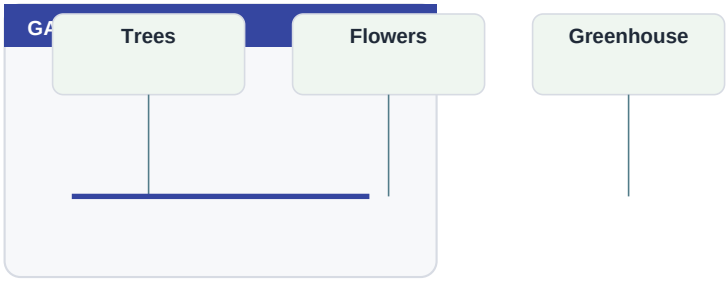
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

70. Temperature Window Pattern

Watering can be restricted by temperature and time.

Garden automation combines water, electricity, weather and time. That makes it useful, but also worth testing carefully with one zone before expanding.



The mental model

Conditions protect plants and hardware.

Where you will use it

- Control tree, flower or greenhouse watering zones with solenoid valves.
- Skip watering when rain, heat or time windows make irrigation unsafe.
- Log every watering decision so you can troubleshoot dry plants or wasted water.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

A safe window reduces evaporation and heat stress.

Common beginner mistake

Forgetting that time needs NTP.

One-minute review

Explain temperature window pattern in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 70: Temperature Window Pattern

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
define MAX_TEMP 35
define MORNING_END 10:00
if (temp < MAX_TEMP && time < MORNING_END) { relay on trees; timer once 900000 relay off trees
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Test the false branch with an impossible threshold.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

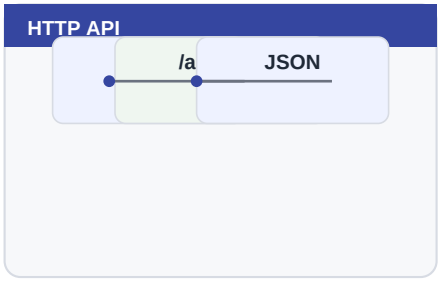
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

71. Daily Health Report

A daily report proves the board is alive.

The API is a bridge to other computers. Start with read-only status calls, then add control only when local safety and logging are already in place.



The mental model

Heartbeat messages are boring until they save a trip.

Where you will use it

- Read status from a laptop or home dashboard.
- Trigger a command from another local automation system.
- Build tests that verify the board is alive after updates.

Terms to remember

- Endpoint: a URL that performs a specific task.
- JSON: structured text returned by the API.
- URL encoding: safe representation of spaces and symbols.

Why this matters

Health reports catch silent failures.

Common beginner mistake

Sending reports too often and training yourself to ignore them.

One-minute review

Explain daily health report in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 71: Daily Health Report

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
function daily_health { if (wifi == connected) mail health "KernelESP daily health" else logger
cron add daily 08:00 call daily_health
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Start with logger if mail is not configured.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

PART 10

Recovery and Release

This part covers Recover from Bad Automation, Serial Rescue Workflow, Safe Boot Thinking and builds toward practical confidence.

Chapters

How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

72. Recover from Bad Automation

Bad cron or scripts should be removable without reflashing.

Recovery is part of design. If a command can start something, another command should let you inspect, pause or undo it.

The mental model

Recovery means pause, inspect, remove, test.

Where you will use it

- Pause automations before debugging.
- Remove a bad cron job without reflashing.
- Use serial as the rescue path when Wi-Fi is down.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

A practiced recovery path makes experimentation safer.

Common beginner mistake

Trying random fixes while automations are still armed.

One-minute review

Explain recover from bad automation in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 72: Recover from Bad Automation

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
disarm
crontab -l
cron rm <id>
timer clear
arm
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Practice disarming before you need it.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

73. Serial Rescue Workflow

Serial rescue is the final trusted path when web access is slow or gone.

Recovery is part of design. If a command can start something, another command should let you inspect, pause or undo it.

The mental model

When the network is uncertain, go local.

Where you will use it

- Pause automations before debugging.
- Remove a bad cron job without reflashing.
- Use serial as the rescue path when Wi-Fi is down.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Many apparent firmware failures are network or script mistakes.

Common beginner mistake

Reflashing before reading logs.

One-minute review

Explain serial rescue workflow in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 73: Serial Rescue Workflow

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
free
wifi status
crontab -l
input list
reboot
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Keep a known-good cable and serial command list nearby.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

74. Safe Boot Thinking

Safe boot keeps startup from becoming a trap.

Recovery is part of design. If a command can start something, another command should let you inspect, pause or undo it.

The mental model

Boot should prepare the system, not perform risky work blindly.

Where you will use it

- Pause automations before debugging.
- Remove a bad cron job without reflashing.
- Use serial as the rescue path when Wi-Fi is down.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Power failures happen; startup behavior must be predictable.

Common beginner mistake

Starting pumps automatically at boot without checks.

One-minute review

Explain safe boot thinking in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 74: Safe Boot Thinking

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
boot show
cat /etc/boot.sh
safe status
dmesg
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Review boot commands after every major automation change.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

75. Firmware Upload

Uploading firmware updates the program while preserving a disciplined workflow.

Recovery is part of design. If a command can start something, another command should let you inspect, pause or undo it.

The mental model

Firmware is the operating brain; assets and files are separate concerns.

Where you will use it

- Pause automations before debugging.
- Remove a bad cron job without reflashing.
- Use serial as the rescue path when Wi-Fi is down.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

A repeatable upload flow avoids mystery states.

Common beginner mistake

Uploading firmware and forgetting web assets.

One-minute review

Explain firmware upload in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 75: Firmware Upload

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
tools/verify.sh
tools/upload.sh /dev/cu.usbserial-XXXX
tools/smoke-http.sh http://<ip> admin
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Never skip verification on a release candidate.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

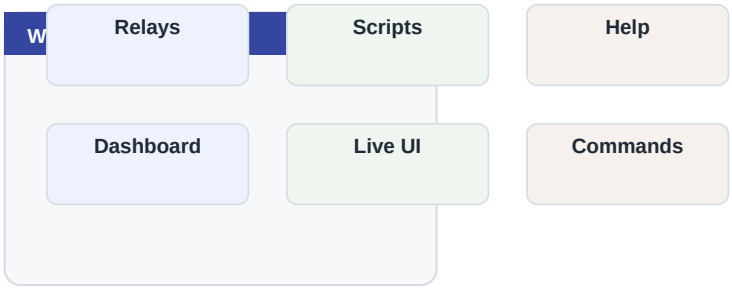
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

76. Asset Upload

Web assets update the browser interface and help files.

The web UI is a friendly window into KernelESP. It should make common work easier, but the underlying commands remain the source of truth.



The mental model

The firmware serves files from /www and /help.

Where you will use it

- Run commands without opening a serial terminal.
- Edit scripts from a browser.
- Read local help when the internet is unavailable.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

UI fixes may live in files, not firmware.

Common beginner mistake

Assuming a firmware flash updates every web file.

One-minute review

Explain asset upload in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 76: Asset Upload

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
tools/upload-assets.sh http://<ip> admin
ls /www
ls /help
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

After asset upload, refresh the browser hard.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

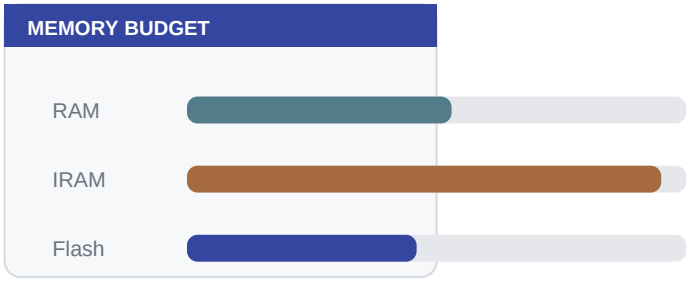
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

77. Release Testing

A release is not done until checks pass.

Memory is a budget. KernelESP can do a lot because it keeps features small and avoids long blocking work.



The mental model

Testing is a ritual that protects future users.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Small devices need boring repeatability.

Common beginner mistake

Calling it done after only compiling.

One-minute review

Explain release testing in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 77: Release Testing

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
tools/verify.sh
tools/smoke-http.sh http://<ip> admin
COUNT=60 DELAY=1 tools/stability-http.sh http://<ip> admin
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Keep test output with your release notes.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

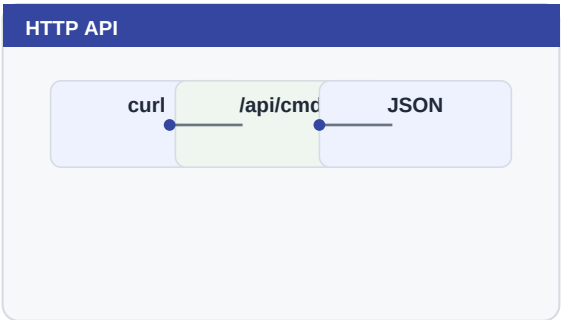
What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

PART 11

Going Further

This part covers Command Reference Habits, API Integrations, Build a Morning Irrigation Program and builds toward practical confidence.

Chapters



How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

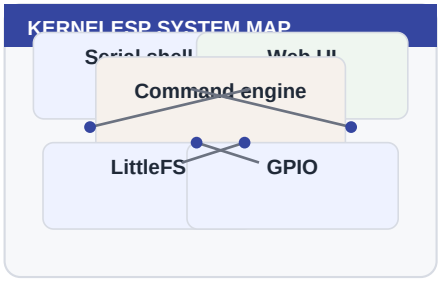
Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

78. Command Reference Habits

The command reference is easier when you read by task, not alphabetically.

System commands are your dashboard. They answer what the board is, what it is doing and whether it has enough resources to continue.



The mental model

Ask: what am I trying to inspect or change?

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Beginners do not need every command at once.

Common beginner mistake

Trying to memorize the whole shell.

One-minute review

Explain command reference habits in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 78: Command Reference Habits

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
help
help cron
help relay
help scripts
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Make your own cheat sheet of ten commands.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

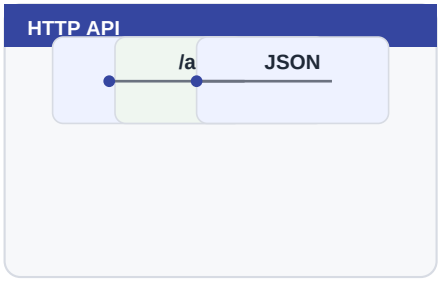
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

79. API Integrations

External systems can call KernelESP without replacing its local control.

The API is a bridge to other computers. Start with read-only status calls, then add control only when local safety and logging are already in place.



The mental model

The API is a bridge, not the boss.

Where you will use it

- Read status from a laptop or home dashboard.
- Trigger a command from another local automation system.
- Build tests that verify the board is alive after updates.

Terms to remember

- Endpoint: a URL that performs a specific task.
- JSON: structured text returned by the API.
- URL encoding: safe representation of spaces and symbols.

Why this matters

Read-only integration is the safest first step.

Common beginner mistake

Letting a remote system send actuator commands before local safety exists.

One-minute review

Explain api integrations in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 79: API Integrations

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
curl -G http://<ip>/api/cmd --data-urlencode key=admin --data-urlencode c='relay status'
curl http://<ip>/api/status?key=admin
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Write one laptop script that reads status only.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

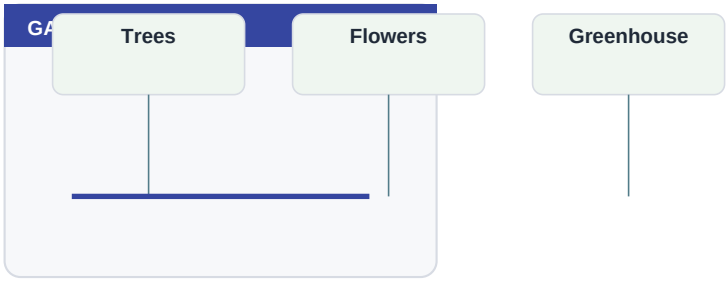
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

80. Build a Morning Irrigation Program

A complete morning program combines time, variables, relays and timers.

Garden automation combines water, electricity, weather and time. That makes it useful, but also worth testing carefully with one zone before expanding.



The mental model

The readable command is the design.

Where you will use it

- Control tree, flower or greenhouse watering zones with solenoid valves.
- Skip watering when rain, heat or time windows make irrigation unsafe.
- Log every watering decision so you can troubleshoot dry plants or wasted water.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Manual first, cron second is the safest habit.

Common beginner mistake

Scheduling a command you have never run manually.

One-minute review

Explain build a morning irrigation program in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 80: Build a Morning Irrigation Program

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
let irrigation.enabled = 1
define WATER_END 10:00
if (irrigation.enabled == 1 && time < WATER_END) { relay on trees; timer once 900000 relay off
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Run it manually before scheduling it.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

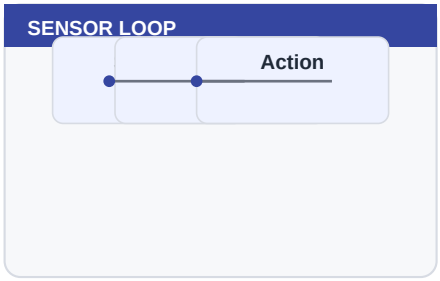
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

81. Build a Temperature Fan Program

Temperature control is a classic automation pattern.

Sensors give the board facts about the environment. Good automation reads those facts, applies simple rules and logs the decision so a human can understand it later.



The mental model

Sense, decide, act, log.

Where you will use it

- Turn on ventilation when temperature rises.
- Send an alert when pressure or humidity crosses a threshold.
- Use hysteresis so equipment does not chatter on and off.

Terms to remember

- I2C: a two-wire data bus used by many sensors.
- Threshold: the value where a rule changes behavior.
- Hysteresis: separate on and off thresholds to avoid chatter.

Why this matters

The same pattern works for heaters, extractors and alarms.

Common beginner mistake

Skipping the off condition.

One-minute review

Explain build a temperature fan program in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 81: Build a Temperature Fan Program

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
define HOT 40
define COOL 35
if (temp >= HOT) { relay on fan; logger fan_on }
if (temp <= COOL) { relay off fan; logger fan_off }
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Use a lamp or LED before a real fan.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

82. Build an Offline Logger

When Wi-Fi is down, logging still gives evidence.

Files make the board persistent. A command changes the current moment; a file lets the board remember a script, configuration or note after reboot.

LITTLEFS LAYOUT

```
/
  /etc      config, cron, relays
  /home     scripts and notes
  /www      web interface
  /var/log  kernel.log
  /func     persistent functions
```

The mental model

Local logs are the fallback memory.

Where you will use it

- Store scripts, installation notes and boot commands.
- Inspect configuration without recompiling firmware.
- Back up the board before experiments.

Terms to remember

- LittleFS: the small flash filesystem used by KernelESP.
- /etc: persistent configuration.
- /home: user scripts and notes.

Why this matters

Good automations degrade gracefully.

Common beginner mistake

Assuming every alert can use the network.

One-minute review

Explain build an offline logger in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 82: Build an Offline Logger

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
if (wifi == connected) { mail health "online" } else { logger offline_no_mail }
tail -n 20 /var/log/kernel.log
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Force a false condition and confirm the local log path.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

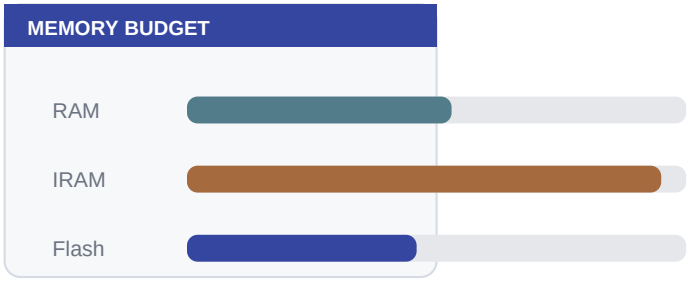
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

83. Keep the System Fast

The ESP8266 web server is small; fewer refreshes and short commands help.

Memory is a budget. KernelESP can do a lot because it keeps features small and avoids long blocking work.



The mental model

Responsiveness is a shared resource.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

A fast UI is often a quiet UI.

Common beginner mistake

Polling many expensive pages every second.

One-minute review

Explain keep the system fast in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 83: Keep the System Fast

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
free
health
ps
top
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Close extra Live UI tabs and compare heap/latency.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

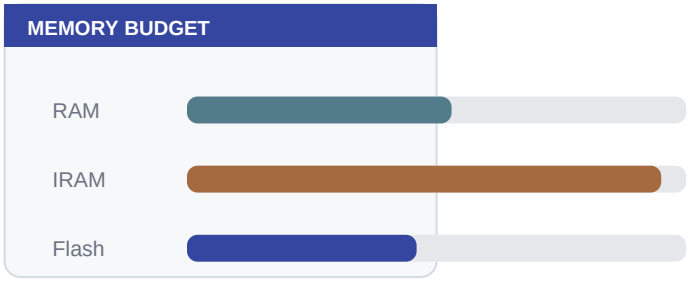
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

84. Know the Limits

KernelESP is powerful because it is small, not because it is unlimited.

Memory is a budget. KernelESP can do a lot because it keeps features small and avoids long blocking work.



The mental model

Limits are design rails.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

Knowing limits prevents accidental complexity.

Common beginner mistake

Treating ESP8266 RAM like a desktop computer.

One-minute review

Explain know the limits in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 84: Know the Limits

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
free
df
timer list
crontab -l
input list
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Write down your board's active relays, timers, inputs and cron jobs.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

85. Document Your Installation

A garden controller needs notes as much as code.

Files make the board persistent. A command changes the current moment; a file lets the board remember a script, configuration or note after reboot.

LITTLEFS LAYOUT

```
/
  /etc      config, cron, relays
  /home     scripts and notes
  /www      web interface
  /var/log  kernel.log
  /func     persistent functions
```

The mental model

Documentation is part of the system.

Where you will use it

- Store scripts, installation notes and boot commands.
- Inspect configuration without recompiling firmware.
- Back up the board before experiments.

Terms to remember

- LittleFS: the small flash filesystem used by KernelESP.
- /etc: persistent configuration.
- /home: user scripts and notes.

Why this matters

Future maintenance starts with knowing what exists.

Common beginner mistake

Relying on memory for outdoor wiring.

One-minute review

Explain document your installation in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 85: Document Your Installation

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
write /home/INSTALL.txt valve1=trees
append /home/INSTALL.txt D1=trees relay
cat /home/INSTALL.txt
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Store pin and valve notes on the board and in your repo.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

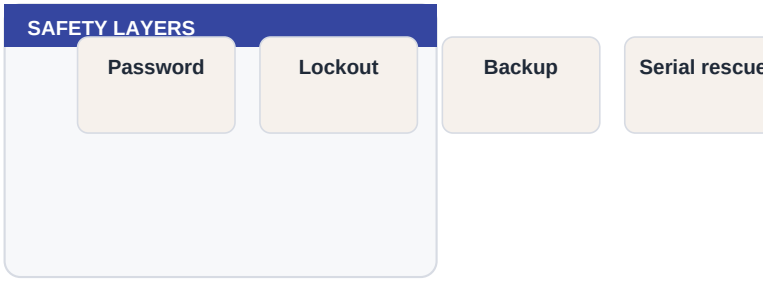
Lab notes

What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

86. Final Beginner Checklist

A safe project is inspected, backed up, tested and recoverable.

Security on a microcontroller is mostly practical discipline: strong keys, private networks, no public exposure and a way to recover locally.



The mental model

The checklist is your calm co-pilot.

Where you will use it

- Use the command manually first.
- Observe the output and logs.
- Only then make the behavior persistent or scheduled.

Terms to remember

- Inspect: read status before changing anything.
- Persist: save the change if it must survive reboot.
- Recover: know the command that pauses or undoes the action.

Why this matters

The best automation is one you can explain and recover.

Common beginner mistake

Leaving a new automation alone without observing one full cycle.

One-minute review

Explain final beginner checklist in one sentence.
Name the command you would run first to inspect this area.
Write one real-world device or project where this idea appears.

Lab 86: Final Beginner Checklist

The goal of this lab is not speed. Type the commands slowly, read the output, and keep the serial console available as your rescue path.

Commands

```
health
free
df
crontab -l
timer list
input list
backup
```

What to check

- You should see a short response, not a frozen shell or a long wait.
- After any command that changes state, run a matching list, status or log command.
- If hardware is involved, repeat the lab once with dryrun or a harmless load.

If it fails

- Read the exact error text before trying another command.
- Run health, free and dmesg to separate resource, network and syntax problems.
- Disarm automations before debugging anything that can move water, heat or mains-powered equipment.

Try this next

Run the checklist before leaving the installation unattended.

What you have learned

You have connected a concept to an observable command. That is the basic KernelESP learning loop: understand the idea, run a small command, inspect the result, then decide whether to make it persistent.

Lab notes

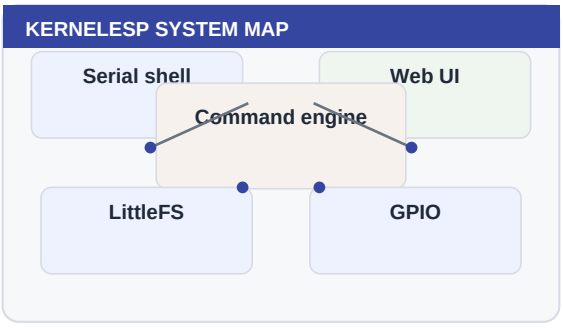
What output did you see?
What command would you run to undo or verify the change?
What would make this lab unsafe on real hardware?

PART 12

Command Atlas

A compact appendix for commands you will use again and again.

Chapters



How to approach it

Read the explanation page first, then do the lab page slowly. The purpose is not to memorize commands, but to build a reliable mental model and a habit of verifying each change.

Beginner promise

You do not have to memorize this part. Run the commands, observe the output, and build confidence one small success at a time.

Before you begin

What do you expect this part to help you control or understand?
Which command would you use to inspect the board before changing anything?

Shell Essentials

These are the commands you use while learning and debugging. They make the board feel like a tiny UNIX shell.

```
help
clear
history
unalias
printenv
unset
false
repeat
man
echo
alias
env
set
true
test
watch
```

System Information

These commands answer the beginner question: is the board healthy, busy or running out of resources?

- `version`
- `uptime`
- `heap`
- `ps`
- `pgrep`
- `kill`
- `reboot`
- `chip`
- `sysinfo`
- `diag`
- `uname`
- `free`
- `mem`
- `top`
- `pidof`
- `dmesg`
- `resetreason`
- `flash`
- `health`

Files and Storage

LittleFS is small, so inspect files and space regularly.

```
pwd
ls
head
grep
wc
stat
write
rm
rmdir
mv
fsformat
cd
cat
tail
find
du
touch
append
mkdir
cp
df
```

Network and Time

Networking and time are the foundation for web access, API integrations, logs and cron.

```
wifi scan
wifi net
wifi connect
wifi reconnect
wifi recover
ifconfig
ap status
date
ntp sync
wifi status
wifi diag
wifi save
wifi wait
wifi sdkreset --yes
ip addr
hostname
ntp kick
```

Automation and Hardware

These are the commands that turn KernelESP from a tiny computer into a controller.

```
relay add
relay off
timer once
cron add
rule add
state set
define
call
when input
mail health
relay on
relay pulse
timer every
crontab -l
scene add
let
function
input add
sensor begin
```

Final Notes

KernelESP is deliberately small. That is its charm and its discipline. The board will reward careful habits: name things well, keep actions short, use timers, log important decisions, back up before experiments and keep a serial rescue path close.

The beginner's rule

If you can inspect it, explain it and undo it, you are ready to automate it.

Before leaving a board unattended

```
health
free
df
crontab -l
timer list
input list
backup
```